# 智能体设计模式

构建智能系统的实战指南[1]，安东尼奥·古利目录 - 共 424 页 = 1+2+1+1+4+9+103+61+34+114+74+5+4 11献词1页致谢，2页[最后，最后一次读取完成]前言，1页[最后，最后一次读取完成]一位思想领袖的视角：权力与责任[最后，最后一次读取完成]介绍，4页[最后，最后一次读取完成]是什么让人工智能系统成为“代理”？，9页[最后，最后一次读取完成]


第一部分（共103页 ）

1.第一章：提示链（代码），12页[最终读取完成，代码正常]2.第二章：路由（代码），13页[完成，上次读取已完成，代码正常3.第三章：并行化（代码），15页[最终，上次读取完成，代码正常4.第四章：反思（代码），13页[最终，上次读取完成，代码正常5.第五章：工具使用（代码 ）），20页[最终读取完成，代码正常6.第六章：规划（代码），13页[最终读取完成，代码正常]7.第七章：多智能体（代码），17页[最终读取完成，代码正常],**121**

第二部分（共61页 ）

8.第八章：内存管理（代码），21页[最终读取完成，代码正常9.第九章：学习与适应（代码），12页[最终读取完成，代码正常10.第十章：模型上下文协议（MCP）（代码），16页[最终读取完成，代码正常11.第十一章：目标设定与监控（代码），12页[最终的，最后读取的，代码 oe],**182**

第三部分（共34页 ）

12.第十二章：异常处理和恢复（代码），8页[最终读取完成，代码正常13.第十三章：人机交互（代码），9页[最终读取完成，代码正常14.第十四章：知识检索（RAG）（代码），17页[最终读取完成，代码正常],**216**

第四部分（共114页 ）

15.第十五章：智能体间通信（A2A））代码），15页[最终读取完成，代码正常16.第十六章：资源感知优化（代码），15页[最终读取完成，代码正常]17.第十七章：推理技巧（代码），24页[最终读取完成，代码正常18.第十八章：护栏/安全模式（代码），19页[最终读取完成，代码正常]19.第十九章：评估与监测（代码），18页[最终读取完成，代码正常]20.第二十章：优先级排序（代码），10页[最终读取完成，代码正常]21.第21章：探索与发现（代码），13页[最终读取完成，代码正常],**330**

附录（共74页 ）

22.附录A：高级提示技巧共28页[最终读取完成，代码正常]23.附录 B - AI 智能体……：从图形用户界面到真实世界环境，6页[最终读取完成，代码正常24.附录 C - 智能体框架快速概述共8页[最终读取完成，代码正常]，25.附录 D - 使用 AgentSpace 构建代理（仅限在线 ）共6页[最终读取完成，代码正常]26.附录 E - CLI 上的 AI 代理（在线 ）共5页[最终读取完成，代码正常]27.附录 F - 深入剖析：智能体的推理引擎内部运作，14页[最终，lrd，代码正常]，28。附录 G - 编码代理共7页**406**结论，5页[最后，最后一次读取完成]词汇表共4页[最后，最后一次读取完成]术语索引，11页（由 *Gemini* 生成。推理步骤包含在智能体示例中。）[最后，lrd]在线贡献 - 常见问题解答：代理设计模式之前通过t：


*https://www.amazon.com/Agentic-Design-Patterns-Hands-Intelligent/dp/3032014018/*

[1]我的所有版税都将捐赠给救助儿童会。

致我的儿子布鲁
诺，

两岁时，他为我的生命带来了崭新而耀眼的光芒。当我探索那些将定义我们未来的体系时，我最关心的始终是你即将继承的世界。

致我的儿子莱昂纳多和洛伦佐，以及我的女儿奥罗拉：

我为你们所成为的优秀女性和男性以及你们正在创造的美好世界感到无比自豪。

本书讲述的是如何构建智能工具，但更饱含着对你们这一代人以智慧和慈悲之心引领这些工具的深切期盼。如果我们学会运用这些强大的技术服务于人类，推动人类进步，那么无论对你们还是对我们所有人来说，未来都将无比光明。

我满怀爱意。

# 致谢

我衷心感谢所有为本书的出版做出贡献的个人和团队。

首先，我要感谢谷歌坚持其使命，赋予谷歌员工权力，并尊重创新的机会。

我感谢首席技术官办公室给予我探索新领域的机会，感谢其秉持“实用魔法”的使命，以及适应新出现的机遇的能力。

我衷心感谢副总裁威尔·格兰尼斯（Will Grannis），感谢他对员工的信任和以身作则的领导风格。感谢我的经理约翰·阿贝尔（John Abel），感谢他鼓励我开展各项工作，并始终以其敏锐的英式洞察力为我提供宝贵的指导。感谢安托万·拉曼贾特（Antoine Larmanjat）在代码层面开发LLM方面的工作，感谢王汉汉（Hann Hann Wang）参与的关于智能体的讨论，以及黄英超（Yingchao Huang）对时间序列的深刻见解。感谢阿什温·拉姆（Ashwin Ram）的领导，感谢玛西·马斯卡罗（Massy Mascaro）的卓越贡献，感谢詹妮弗·贝内特（Jennifer Bennett）的技术专长，感谢布雷特·斯拉特金（Brett Slatkin）的工程支持，以及埃里克·申（Eric Schen）的启发性讨论。OCTO团队，特别是斯科特·彭伯西（Scott Penberthy），值得特别表扬。最后，衷心感谢帕特里夏·弗洛里西（Patricia Florissi）对智能体社会影响的远见卓识。

我还要感谢 Marco Argenti，他提出了富有挑战性和激励性的愿景，即利用经纪人来增强人力资源。同时，我也要感谢 Jim Lanzone 和 Jordi Ribas，他们不断提升搜索领域和经纪人

领域之间的关系。

我还要感谢云端人工智能团队，特别是他们的领导 Saurabh Tiwary，感谢他带领人工智能组织朝着有原则的方向发展。感谢区域技术负责人 Salem Haykal，他是一位鼓舞人心的同事。感谢 Google Agentspace 的联合创始人 Vladimir Vuskovic，感谢 Kate (Katarzyna) Olszewska 在 Kaggle Game Arena 上的 Agentic 合作，感谢 Nate Keating 对 Kaggle 的热情投入，Kaggle 社区为人工智能做出了巨大贡献。还要感谢 Kamelia Aryafa，她领导着专注于 Agentspace 和 Enterprise NotebookLM 的应用人工智能和机器学习团队；以及 Jahn Wooland，他是一位真正以交付成果为导向的领导者，也是一位随时提供建议的挚友。

特别感谢黄英超，他是一位才华横溢的人工智能工程师，前途无量；还要感谢王瀚，是他激励我重拾对智能体的兴趣。[1]
1994 年的初步兴趣，以及 Lee Boonstra 在快速工程方面做出的杰出贡献。

我还要感谢 GenAI 五日活动团队，包括副总裁 Alison Wagonfeld 对团队的信任，Anant Nawalgaria 一如既往的出色表现，以及 Paige Bailey 的积极态度和领导才能。

我还要深深感谢 Mike Styer、Turan Bulmus 和坎查纳·帕特洛拉感谢您帮助我在 Google I/O 2025 大会上发布了三个代理程序。非常感谢您付出的巨大努力。

我衷心感谢 Thomas Kurian 在推动云计算和人工智能项目方面展现出的坚定领导力、热情和信任。我也由衷感谢 Emanuel Taropa，他积极乐观、勇于担当的态度使他成为我在谷歌遇到的最杰出的同事，为我树立了深刻的榜样。最后，感谢 Fiona Cicconi 与我进行的关于谷歌的精彩讨论。

我衷心感谢 Demis Hassabis、Pushmeet Kohli 以及整个 GDM 团队，感谢他们为开发 Gemini、AlphaFold、AlphaGo 和 AlphaGenome 等项目所付出的热情努力，以及他们为推动科学进步、造福社会所做出的贡献。特别感谢 Yossi Matias 对 Google Research 的领导，以及他一直以来提供的宝贵建议。我从您身上学到了很多。

特别感谢帕蒂·梅斯，她在90年代率先提出了软件代理的概念，并始终致力于研究计算机系统和数字设备如何增强人类能力，帮助他们解决记忆、学习、决策、健康和福祉等方面的问题。您在91年的愿景如今已成为现实。

我还要感谢保罗·德鲁加斯和施普林格出版社的所有出版团队成员，感谢他们让这本书得以出版。

我由衷感谢众多才华横溢的人士，是他们的帮助才成就了这本书的诞生。我衷心感谢

Marco Fago 的巨大贡献，他不仅编写了代码和图表，还审阅了全文。我也感谢 Mahtab Syed 的编码工作，以及 Ankita Guha 对众多章节提出的极其详尽的反馈。Priya Saxena 的精辟修改、Jae Lee 的细致审阅，以及 Mario da Roza 为创建 NotebookLM 版本所做的辛勤工作，都极大地提升了本书的质量。我非常幸运地拥有一支专家审阅团队，他们为本书的最初章节提供了宝贵的意见，在此我感谢 Amita Kapoor 博士、Fatma Tarlaci 博士、Alessandro Cornacchia 博士和 Aditya Mandlekar 的专业知识。我还要真诚地感谢 Ashley Miller、A Amir John 和 Palak Kamdar (Vasani) 的独特贡献。最后，我要衷心感谢 Rajat 一直以来的坚定支持和鼓励。2
Jain、Aldo Pahor、Gaurav Verma、Pavithra Sainath、Mariusz Koczwara、Abhijit Kumar、Armstrong Foundjem、Haiming Ran、Udita Patel 和 Kaurnakar Kotha。

如果没有你，这个项目根本不可能完成。所有的功劳都归功于你，所有的错误都由我承担。

我的所有版税都捐给了救助儿童会。

# 前言

人工智能领域正处于一个激动人心的转折点。我们正在超越构建能够简单处理信息的模型，转向创建能够推理、规划和行动的智能系统，以完成具有模糊性任务的复杂目标。正如本书恰如其分地描述的那样，这些“智能体”系统代表了人工智能的下一个前沿领域，它们的开发对于谷歌而言既是一项挑战，也令我们感到兴奋和鼓舞。

《智能体设计模式：构建智能系统的实战指南》的出版恰逢其时，为我们开启了这段旅程。本书一针见血地指出，大型语言模型（即这些智能体的认知引擎）的强大功能必须通过结构化和周密的设计来驾驭。正如设计模式通过提供通用语言和可重用的解决方案来革新软件工程一样，本书中的智能体模式也将成为构建健壮、可扩展且可靠的智能系统的基础。

用“画布”来比喻构建智能体系统，与我们在谷歌Vertex AI平台上的工作理念不谋而合。我们致力于为开发者提供最强大、最灵活的画布，助力他们构建下一代人工智能应用。本书提供实用且易于操作的指导，帮助开发者充分发挥这块画布的潜力。本书探讨了从提示链和工具使用到智能体间协作、自我纠错、安全性和防护机制等各种模式，为希望构建复杂人工智能智能体的开发者提供了一套全面的工具包。

人工智能的未来将取决于能够构建这些智能系统的开发者的创造力和独创性。《智能体设计模式》是一本不可或缺的资源，它将帮助激发这种创造力。本书不仅提供了理解智能体系统的“是什么”和“为什么”，还提供了理解其“如何做”所需的基本知识和实践案例。

我很高兴看到这本书来到开发者社区。书中阐述的模式和原则无疑将加速开发创新且具有深远影响的人工智能应用，这些应用将在未来数年内塑造我们的世界。

Saurabh Tiwari，谷歌 CloudAI 副总裁兼总经理

# 一位思想领袖的视角：权力与责任

在过去四十年里，我见证了无数的技术周期——从个人电脑和互联网的诞生，到移动和云计算的革命——但没有一个周期像现在这样截然不同。多年来，围绕人工智能的讨论一直遵循着炒作与幻灭的熟悉节奏，所谓的“人工智能夏季”之后便是漫长而寒冷的冬季。但这一次，情况有所不同。讨论的焦点已经发生了显著的转变。如果说过去十八个月的重点是引擎——大型语言模型（LLM）令人叹为观止、近乎垂直的崛起——那么下一个时代将围绕我们围绕它构建的“汽车”展开。它将围绕着如何驾驭这种原始力量的框架展开，将其从一个生成看似合理的文本的工具转变为一个真正的行动主体。

我承认，起初我持怀疑态度。我发现，可信度往往与一个人对某个主题的了解程度成反比。早期

的模型虽然流畅，但总感觉像是带着某种冒名顶替综合症在运作，它们追求的是可信度而非正确性。但随后出现了转折点，一种新型“推理”模型带来了质的飞跃。突然间，我们不再只是在与一台预测序列中下一个词的统计机器对话；我们得以窥见一种萌芽状态的认知形式。

第一次尝试使用这种新型智能编码工具时，我感受到了那种熟悉的魔力。我把它交给它处理一个我一直没时间做的个人项目：将一个慈善机构的网站从一个简单的网页构建器迁移到一个完善的、现代化的 CI/CD 环境。接下来的二十分钟里，它开始工作，提出澄清问题，请求凭据，并提供状态更新。感觉不像是在使用工具，而更像是在与一位初级开发人员协作。当它向我交付一个完全可部署的软件包，并附带完善的文档和单元测试时，我简直惊呆了。

当然，它并不完美。它会犯错，会卡住。它需要我的监督，更重要的是，需要我的判断才能让它重回正轨。这次经历让我深刻体会到我在漫长的职业生涯中艰难领悟到的一个教训：你不能盲目信任。然而，这个过程却令人着迷。窥探它的“思维链”就像观看一个大脑的运作——混乱、非线性，充满了开始、停止和自我修正，与我们人类的推理过程颇为相似。它不是一条直线；而是一次通往解决方案的随机漫步。这其中蕴藏着某种新事物的雏形：它不仅是一种能够生成内容的智能，更是一种能够生成……的智能。计划。

这就是智能体框架的优势所在。它就像静态的地铁线路图和能够实时重新规划路线的动态GPS之间的区别。传统的基于规则的自动机遵循固定路径；当它遇到意外障碍时，就会崩溃。而由推理模型驱动的人工智能智能体则具备观察、适应和寻找其他路径的潜力。它拥有某种数字常识，使其能够应对现实中无数的极端情况。
这标志着从简单地告诉计算机转变为另一种方式。什么 去做，去解释为什么 我们需要做点什么，并且相信它能找到解决办法。如何。

尽管这片新领域令人兴奋，但也带来了一份沉甸甸的责任感，尤其对我这个全球金融机构的首席信息官而言更是如此。风险极其巨大。一个代理人在制作“鸡肉三文鱼融合派”的食谱时出错，或许只是个有趣的轶事。但如果代理人在执行交易、管理风险或处理客户数据时出错，那就成了真正的问题。我读过相关的免责声明和警示案例：比如，一个网络自动化代理在登录失败后，竟然给一位议员发邮件抱怨登录障碍。这则略带黑色幽默的案例提醒我们，我们正在面对一项我们尚未完全了解的技术。

在这里，工艺、文化以及对原则的不懈坚持成为我们最重要的指引。我们的工程准则不仅仅是纸上的文字，它们是我们的指南针。我们必须以目标为导向进行建设确保我们设计的每个代理都从对客户问题的清晰理解出发，而这些问题正是我们需要解决的。我们必须环顾四周预测故障模式并设计具有弹性的系统。最重要的是，我们必须激发信任通过公开透明的方法和对结果负责。

在智能体主导的世界里，这些原则显得尤为紧迫。残酷的现实是，你不能简单地将这些强大的新工具叠加到混乱且不一致的系统中，就指望获得良好的结果。混乱的系统加上智能体，无异于自取灭亡。用“垃圾”数据训练的人工智能不仅会产生垃圾结果，还会产生看似合理、充满自信的垃圾结果，从而毒害整个流程。因此，我们首要且至关重要的任务是做好准备工作。我们必须投资于干净的数据、一致的元数据和定义完善的API。我们必须构建现代化的“跨州系统”，使这些智能体能够安全、高速地运行。这是一项艰巨的基础性工作，旨在构建一个可编程的企业，一个“企业即

软件”，在这个企业中，我们的流程架构与代码架构一样精良。

归根结底，这段旅程并非要取代人类的创造力，而是要增强它。它要求我们所有人掌握一系列新的技能：清晰地阐述任务的能力、明智的授权技巧以及认真核实成果质量的严谨态度。它要求我们保持谦逊，承认自身的不足，并永不止步地学习。本书接下来的内容将提供构建这些新框架的技术路线图。我希望你们不仅能运用这些技术构建可能实现的框架，更能构建正确、稳健且负责任的框架。

世界呼吁每一位工程师挺身而出。我相信我们已经准备好迎接挑战。

享受旅程。

Marco Argenti，高盛首席信息官

# 序言

欢迎阅读《智能体设计模式：构建智能系统的实战指南》。纵观现代人工智能的发展历程，我们可以看到一个清晰的演进过程：从简单的被动程序，发展到能够理解上下文、做出决策并与环境和其他系统动态交互的复杂自主实体。这些就是智能体以及它们所构成的智能体系统。

强大的大型语言模型（LLM）的出现，为理解和生成类人内容（例如文本和媒体）提供了前所未有的能力，并成为许多智能体的认知引擎。然而，要将这些能力整合到能够可靠地实现复杂目标的系统中，需要的不仅仅是一个强大的模型。它还需要结构、设计，以及对智能体如何感知、计划、行动和交互的深思熟虑的方法。

把构建智能系统想象成在画布上创作一件复杂的艺术作品或工程作品。这块画布并非空白的视觉空间，而是为智能体提供生存和运行环境及工具的底层基础设施和框架。它是你构建智能应用程序的基础，用于管理状态、通信、工具访问和逻辑流程。

在这个智能体画布上进行有效构建，需要的不仅仅是简单地将各个组件拼凑在一起，还需要理解已被验证有效的技术——图案——旨在解决设计和实现智能体行为时遇到的常见挑战。正如建筑模式指导建筑施工，设计模式构建软件结构一样，智能体设计模式为在您选择的平台上实现智能体时遇到的反复出现的问题提供了可重用的解决方案。

## 什么是智能体系统？

从本质上讲，智能体系统是一种计算实体，它能够感知其环境（包括数字环境和潜在的物

理环境），基于这些感知以及一组预定义或学习到的目标做出明智的决策，并自主执行行动以实现这些目标。与遵循僵化、循序渐进指令的传统软件不同，智能体展现出一定程度的灵活性和主动性。

假设你需要一个系统来管理客户咨询。传统的系统可能遵循固定的脚本。然而，智能体系统可以感知客户查询的细微差别，访问知识库，并与其他内部系统（例如……）进行交互。



（例如订单管理），可能会提出澄清问题，并主动解决问题，甚至可能预测未来的需求。这些代理程序在您的应用程序基础架构上运行，利用可用的服务和数据。

智能体系统通常具有以下特征：自治使他们能够在没有持续的人工监督的情况下行动；主动性并开始采取行动来实现他们的目标；反应性能够有效应对环境变化。它们本质上是目标导向不断朝着目标努力。一项关键能力是工具使用这使它们能够与外部 API、数据库或服务进行交互——有效地拓展到它们直接影响范围之外。它们拥有记忆能够跨交互保留信息，并能参与沟通 与在同一画布或连接的画布上运行的用户、其他系统，甚至其他代理。

有效实现这些特性会带来显著的复杂性。智能体如何在画布上的多个步骤中保持状态？它如何做出决定？什么时候 和 如何 如何使用该工具？如何管理不同代理之间的沟通？如何增强系统的弹性以应对意外结果或错误？

# 为什么模式在智能体开发中至关重要

正是这种复杂性使得智能体设计模式不可或缺。它们并非僵化的规则，而是经过实战检验的模板或蓝图，为智能体领域中常见的设计和实现挑战提供了行之有效的解决方案。通过识别和应用这些设计模式，您可以获得能够提升您在画布上构建的智能体的结构、可维护性、可靠性和效率的解决方案。

使用设计模式可以帮助您避免重复发明诸如管理对话流程、集成外部功能或协调多个代理操作等基本解决方案。它们提供了一种通用语言和结构，使您的代理逻辑更清晰，更易于他人（以及您自己）理解和维护。实施专为错误处理或状态管理而设计的模式，有助于构建更健壮、更可靠的系统。利用这些成熟的方法可以加快您的开发流程，使您能够专注于应用程序的独特方面，而不是代理行为的基础机制。

本书提取了 21 种关键设计模式，这些模式代表了在各种技术平台上构建复杂代理的基

本构建模块和技术。



理解和应用这些模式将显著提高您有效设计和实施智能系统的能力。

# 本书概述及使用方法

这本书，“智能体设计模式《构建智能系统的实战指南》旨在成为一本实用且易于理解的参考书。本书主要侧重于清晰地解释每一种智能体模式，并提供具体、可运行的代码示例来演示其实现。全书共21章，涵盖了各种设计模式，从构建顺序操作（提示链 ）和外部交互（工具使用）等基础概念，到协作工作（多智能体协作）和自我改进（自我纠正）等更高级的主题。

本书按章节组织，每章深入探讨一种主体模式。每章内容包括：

- 详细的模式概述对这种模式及其在智能设计中的作用进行了清晰的解释。
- 关于……的部分实际应用及案例举例说明这种模式在现实世界中的价值及其带来的好处。
- A动手实践代码示例提供实用且可运行的代码，演示如何使用主流的代理开发框架来实现该模式。在这里，您将看到如何在技术画布的上下文中应用该模式。
- 要点总结总结要点，方便快速回顾。
- 参考 为了进一步探索，提供资源以便更深入地学习该模式及相关概念。

本书各章节的编排顺序旨在循序渐进地构建概念，但您也可以将其作为参考资料，直接跳转到解决您在智能体开发项目中遇到的具体挑战的章节。附录全面介绍了高级提示技术、在真实环境中应用人工智能智能体的原则，以及基本的智能体框架概述。此外，本书还包含实用的在线教程，提供使用 AgentSpace 等特定平台以及命令行界面构建智能体的分步指导。本书始终强调实际应用；我们强烈建议您运行代码示例，进行实验，并根据您选择的平台进行调整，从而构建您自己的智能系统。



我经常听到一个很好的问题：“人工智能发展如此迅速，为什么还要写一本可能很快就会过

时的书呢？”我的初衷恰恰相反。正因为事物发展如此迅速，我们才更需要停下来，找出那些正在逐渐成型的基本原理。像红绿灯（RAG）、反射、路由、内存以及我在书中讨论的其他模式，正在成为构建人工智能的基本模块。这本书旨在邀请读者反思这些核心理念，它们为我们构建人工智能提供了基础。人类需要这些对基础模式的反思时刻。

## 所用框架简介

为了给我们的代码示例提供一个切实的“画布”（另见附录），我们将主要利用三个杰出的代理开发框架。朗链及其有状态扩展**LangGraph**它提供了一种灵活的方式将语言模型和其他组件链接在一起，为构建复杂的操作序列和图提供了一个强大的平台。船员人工智能它提供了一个结构化的框架，专门用于协调多个人工智能代理、角色和任务，作为一个平台，尤其适合协作代理系统。**Google Agent Developer Kit**（**Google ADK**）提供用于构建、评估和部署代理的工具和组件，提供另一个有价值的平台，通常与谷歌的 AI 基础设施集成。

这些框架代表了智能体开发画布的不同方面，各有优势。通过展示这些工具的示例，您可以更全面地了解如何应用这些模式，而无需考虑您为智能体系统选择的具体技术环境。这些示例旨在清晰地阐释模式的核心逻辑及其在框架画布上的实现，重点在于清晰性和实用性。

读完本书，您不仅将理解 21 种基本智能体模式背后的基本概念，还将掌握有效应用这些模式的实践知识和代码示例，从而能够在您选择的开发平台上构建更智能、更强大、更自主的系统。让我们开始这段实践之旅吧！



# 是什么让人工智能系统成为智能体？

简单来说，人工智能代理智能体人工智能（Agentic AI）是一个旨在感知环境并采取行动以实现特定目标的系统。它是标准大型语言模型（LLM）的演进版本，增强了规划、使用工具

和与周围环境交互的能力。可以将智能体人工智能想象成一个在工作中不断学习的智能助手。它遵循一个简单的五步循环来完成任务（见图1）：

1.获取任务：1. 你给它设定一个目标，比如“安排我的日程”。2.扫描场景：它收集所有必要的信息——读取电子邮件，

查看日历和访问联系人——以了解正在发生的事情。3.仔细思考：它通过考虑最优因素来制定行动计划

实现目标的方法。4.采取行动：它通过发送邀请、安排会议等方式执行计划。

并更新您的日历。5.学习和进步：它会观察成功案例并做出相应调整。例如，如果会议改期，系统会从中吸取经验教训，以提升未来的表现。



图1：智能体人工智能作为智能助手，通过不断的经验学习。它通过一个简单的五步循环来完成任务。

人工智能代理正以惊人的速度迅速普及。根据近期研究，大多数大型IT公司都在积极使用这些代理，其中五分之一的公司是在过去一年内才开始使用。金融市场也注意到了这一

点。到2024年底，人工智能代理初创公司已筹集超过20亿美元资金，市场估值达到52亿美元。预计到2034年，该市场规模将爆炸式增长至近2000亿美元。简而言之，种种迹象表明，人工智能代理将在未来的经济中扮演举足轻重的角色。

短短两年内，人工智能范式发生了翻天覆地的变化，从简单的自动化发展到复杂的自主系统（见图 2）。最初，工作流程依赖于基本的提示和触发器来处理使用逻辑学习模型（LLM）的数据。随着检索增强生成（RAG）技术的出现，这种方法得到了进一步发展，RAG 通过将模型建立在事实信息之上，提高了可靠性。随后，我们见证了能够使用各种工具的独立人工智能代理的出现。如今，我们正迈入智能体人工智能时代，在这个时代，由多个专业代理组成的团队协同工作，以实现复杂的目标，这标志着人工智能协作能力的显著提升。



图 2：从 LLM 过渡到 RAG，再到 Agentic RAG，最后到 Agentic AI。

本书旨在探讨专业代理如何协同工作以实现复杂目标的设计模式，每一章都会介绍一种协作和交互的范式。

在此之前，让我们先来看一些涵盖不同智能体复杂程度的例子（见图 3）。

## 第 0 级：核心推理引擎

虽然逻辑逻辑模型（LLM）本身并非智能体，但它可以作为基础智能体系统的推理核心。在“0级”配置下，LLM无需工具、记忆或环境交互，完全基于其预训练知识做出反应。它的优势在于能够利用其庞大的训练数据来解释既定概念。这种强大的内部推理能力带来的代价是完全缺乏对当前事件的感知。例如，如果2025年奥斯卡最佳影片的获奖信息超出其预训练知识范围，它就无法说出获奖影片。

## 第一级：关联问题解决者

在这个层面上，LLM 通过连接和利用外部工具，成为一个功能完善的智能体。它的解决问题能力不再局限于预训练的知识。相反，它可以执行一系列操作，从互联网（通过搜索）或数据库（通过检索增强生成，即 RAG）等来源收集和处理信息。更多详细信息，请参阅第 14 章。

例如，为了找到新的电视节目，智能体会识别出对最新信息的需求，使用搜索工具查找信息，然后综合分析结果。至关重要的是，它还可以使用专门的工具来提高准确性，例如调用金融API来获取苹果公司（AAPL）的实时股价。这种与外部世界进行多步骤交互的能力是1级智能体的核心能力。

## 第二级：战略问题解决者

在这个级别，代理的能力显著扩展，包括战略规划、主动协助和自我改进，而快速工程和情境工程是核心使能技能。

首先，智能体不再局限于使用单一工具，而是通过策略性问题解决来应对复杂的多部分问题。在执行一系列动作的过程中，它会积极主动地进行策略性思考和行动。



执行上下文工程：这是一个策略性的过程，旨在为每个步骤选择、打包和管理最相关的信息。例如，要在两个地点之间找到一家咖啡店，它首先使用地图工具。然后，它对地图输出进行处理，筛选出一个简短、聚焦的上下文（例如，可能只是街道名称列表），并将其输入到本地搜索工具中，从而避免认知过载，确保第二步高效准确。为了使人工智能达到最高的准确率，必须为其提供简短、聚焦且有效的上下文。上下文工程正是通过策略性地选择、

打包和管理来自所有可用来源的最关键信息来实现这一目标的学科。它有效地管理模型有限的注意力，防止认知过载，并确保在任何给定任务上都能高效、高质量地完成任务。更多详细信息，请参阅附录 A。

这种级别能够实现主动且持续的操作。与您的电子邮件关联的旅行助手就体现了这一点：它能够从冗长的航班确认邮件中提取上下文信息，并仅选取关键细节（航班号、日期、地点）进行打包，以便后续调用工具访问您的日历和天气 API。

在软件工程等专业领域，代理程序通过应用这种方法来管理整个工作流程。当分配到缺陷报告时，它会读取报告并访问代码库，然后策略性地将这些庞大的信息源整合到一个高效且重点突出的上下文中，从而能够高效地编写、测试和提交正确的代码补丁。

最后，智能体通过不断改进自身的上下文工程流程来实现自我提升。当它请求关于如何改进提示的反馈时，它实际上是在学习如何更好地整理初始输入。这使其能够自动改进未来任务的信息打包方式，从而形成一个强大的自动化反馈循环，随着时间的推移不断提高其准确性和效率。更多详细信息，请参阅第17章。

图 3:各种实例展示了代理复杂性的谱系。

## 第三级:协作式多智能体系统的兴起

在第三阶段,我们看到人工智能发展发生了重大范式转变,不再追求单一的、无所不能的超级智能体,而是转向构建复杂、协作的多智能体系统。本质上,这种方法认识到,解决复杂挑战的最佳方案往往不是由单个通才,而是由一群专家协同合作。这种模型直接反映了人类组织的结构,其中不同的部门被赋予特定的角色,并协作应对多方面的目标。这种系统的整体力量源于这种分工以及通过协调努力所产生的协同效应。更多详细信息,请参阅第七章。

为了更好地理解这个概念,不妨设想一下新产品上市的复杂流程。与其让一位专员试图包揽所有环节,不如设立一位“项目经理”作为中央协调人。这位经理将统筹整个流程,把任务分配给其他专业专员:例如,“市场调研”专员负责收集消费者数据,“产品设计”专员负责产品概念开发,“市场营销”专员负责制作宣传材料。他们成功的关键在于彼此之间顺畅的沟通和信息共享,确保所有人的努力都朝着共同的目标前进。



尽管这种基于团队的自主自动化愿景已在发展之中,但我们必须正视当前面临的挑战。此类多智能体系统的有效性目前受限于其所使用的逻辑逻辑模型(LLM)的推理能力。此外,

它们之间真正相互学习并作为一个整体不断改进的能力仍处于起步阶段。克服这些技术瓶颈是至关重要的下一步，而这将释放这一层面的巨大潜力：实现从头到尾的整个业务流程的自动化。

## 智能体的未来：五大假设

人工智能代理的开发正以前所未有的速度在软件自动化、科学研究和客户服务等领域蓬勃发展。尽管目前的系统令人印象深刻，但这仅仅是个开始。下一波创新浪潮很可能聚焦于提升代理的可靠性、协作性和与我们生活的深度融合。以下是关于未来发展的五大主要假设（见图 4）。

## 假设1：通才型主体的出现

第一个假设是，人工智能代理将从专精于特定领域的专家进化为真正的通才，能够以高可靠性管理复杂、模糊且长期的目标。例如，您可以给代理一个简单的指令："为我公司下季度在里斯本举办的30人团建活动做计划。" 代理随后将管理整个项目数周，处理从预算审批、机票谈判到场地选择、根据员工反馈制定详细行程等所有事宜，并定期更新进度。要实现这种程度的自主性，需要在人工智能推理、记忆和近乎完美的可靠性方面取得根本性的突破。另一种方法（并非与前者互斥）是小型语言模型（SLM）的兴起。这种"乐高式"的概念是指用小型、专业的专家代理来构建系统，而不是扩展单个庞大的模型。这种方法有望带来更便宜、调试更快、部署更便捷的系统。最终，开发大型通才模型和构建小型专业模型都是可行的发展路径，它们甚至可以相互补充。

## 假设2：深度个性化和主动目标发现

第二个假设认为，智能体将成为高度个性化且积极主动的合作伙伴。我们正在见证一种新型智能体的出现：积极主动的合作伙伴。通过学习您独特的行为模式和目标，这些系统正从仅仅执行指令转变为预测您的需求。人工智能系统



当它们不再仅仅是回复聊天信息或指令时，它们就扮演了代理人的角色。它们代表用户发起并执行任务，并在过程中积极协作。这超越了简单的任务执行，进入了主动发现目标的领域。

例如，如果您正在探索可持续能源，智能助手可能会识别出您的潜在目标，并通过推荐课程或总结研究成果来主动提供支持。虽然这些系统仍在发展中，但其发展轨迹清晰可

见。它们将变得越来越主动，并在确信采取行动对您有益时，学会代表您主动行事。最终，智能助手将成为您不可或缺的盟友，帮助您发现并实现那些您尚未完全表达出来的抱负。

图 4：关于代理人未来的五种假设

## 假设3：具身认知与物理世界互动

这一假设预见了智能体将突破纯粹的数字限制，在物理世界中运行。通过将智能体人工智能与机器人技术相结合，我们将看到“具身智能体”的兴起。届时，你或许不再只是预约一个杂工，而是可以请你的智能体来修理漏水的水龙头。智能体将利用其视觉传感器来感知问题，7
用户可以访问管道知识库来制定方案，然后精确控制机械臂进行维修。这将是一项里程碑式的进步，它弥合了数字智能与物理操作之间的鸿沟，并将彻底改变从制造业和物流到老年护理和家庭维护等各个领域。

## 假设4：主体驱动经济

第四个假设是，高度自主的智能体将成为经济活动的积极参与者，创造新的市场和商业模式。我们可以设想，智能体作为独立的经济实体，以最大化特定目标（例如利润）为己任。企业家可以启动一个智能体来运营整个电子商务业务。该智能体将通过分析社交媒体来识别热门产品，生成营销文案和视觉素材，通过与其他自动化系统交互来管理供应链物流，并根据实时需求动态调整价格。这种转变将创造一个全新的、超高效的“智能体经济”，其运行速度和规模是人类直接管理的极限。

### 假设5：目标驱动的变形多智能体系统

这一假设提出，智能系统将不再依赖显式编程，而是根据预设目标运行。用户只需陈述期望的结果，系统便能自主地找到实现目标的方法。这标志着智能系统向能够真正实现个体和群体自我完善的、可变形的多智能体系统发生了根本性的转变。

该系统将是一个动态实体，而非单一主体。它能够分析自身性能，并根据需要调整其多主体团队的拓扑结构，创建、复制或移除主体，从而组建最高效的团队来完成当前任务。这种演化发生在多个层面：

- 架构修改：从最深层次上看，各个代理可以重写自己的源代码，并重新构建其内部结构以提高效率，正如最初的假设一样。
- 指令修正：在更高层面上，系统持续进行自动提示工程和上下文工程。它优化发送给每个代理的指令和信息，确保它们在无需人工干预的情况下获得最佳指导。

例如，一位创业者只需声明其意图：“创办一家成功的电子商务企业，销售手工咖啡。”系统无需进一步编程，即可立即启动。它最初可能会生成一个“市场调研”代理和一个“品牌推广”代理。根据初步调查结果，系统可能会决定移除某个代理。



品牌代理会衍生出三个新的专业代理：“Logo设计”代理、“网店平台”代理和“供应链”代理。系统会不断调整这些代理的内部指令，以提升性能。如果网店代理成为瓶颈，系统可能会将其复制成三个并行代理，分别负责网站的不同部分，从而动态地重构自身结构，以更好地实现既定目标。

## 结论

本质上，人工智能代理代表着对传统模型的重大飞跃，它作为一个自主系统，能够感知、规划并采取行动以实现特定目标。这项技术的发展正从单一的、使用工具的代理，逐步迈向

能够处理多方面目标的复杂协作式多代理系统。未来的设想预测，通用型、个性化甚至实体化的代理将会出现，并成为经济活动的积极参与者。这一持续发展标志着一次重大的范式转变，即朝着自我改进、目标驱动的系统迈进，这些系统有望实现整个工作流程的自动化，并从根本上重新定义我们与技术的关系。

## 参考


1. Cloudera公司（2025年4月）报告称，96%的企业正在增加人工智能的使用。代理人。https://www.cloudera.com/about/news-and-blogs/press-releases/2025-04-16-96-percent-of-enterprises-are-expanding-use-of-ai-agents-according-to-latest-data-from-cloudera.html2. 自主生成式人工智能代理：https://www.deloitte.com/us/en/insights/industry/technology/technology-media-and-telecom-predictions/2025/autonomous-generative-ai-agents-still-under-development.html3. Market.us. 全球智能体人工智能市场规模、趋势和预测（2025-2034 年）。https://market.us/report/agentic-ai-market/




# 第一章：提示链

## 提示链模式概述

提示链（有时也称为管道模式）是一种强大的范式，可用于处理利用大型语言模型 (LLM) 时遇到的复杂任务。提示链并不期望 LLM 一次性解决复杂问题，而是提倡分而治之的策略。其核心思想是将最初看似棘手的问题分解成一系列更小、更易于管理的子问题。每个子问题都通过专门

设计的提示来单独解决，并且一个提示的输出会策略性地作为输入传递给链中的下一个提示。

这种顺序处理技术从本质上为与逻辑逻辑模型（LLM）的交互引入了模块化和清晰度。通过分解复杂任务，可以更轻松地理解和调试每个步骤，从而使整个过程更加稳健和易于解释。链中的每个步骤都可以精心设计和优化，以专注于更大问题的特定方面，从而获得更准确、更有针对性的输出。

前一步骤的输出作为下一步骤的输入至关重要。这种信息传递建立了一个依赖链（这也是“依赖链”名称的由来），其中先前操作的上下文和结果指导着后续的处理。这使得LLM能够在前期工作的基础上不断完善理解，并逐步接近理想的解决方案。

此外，提示链式推理不仅能分解问题，还能整合外部知识和工具。在每个步骤中，LLM 都可以被指示与外部系统、API 或数据库交互，从而拓展其知识和能力，使其超越内部训练数据。这种能力极大地扩展了 LLM 的潜力，使其不仅能作为独立模型运行，还能作为更广泛、更智能系统的组成部分发挥作用。

提示链的意义远不止于简单的解决问题。它是一种构建复杂人工智能体的基础技术。这些智能体可以利用提示链在动态环境中自主地进行规划、推理和行动。通过策略性地构建提示序列，智能体可以执行需要多步骤推理、规划和决策的任务。这种智能体的工作流程能够更贴近人类的思维过程，从而实现与复杂领域和系统更自然、更高效的交互。



单次提示的局限性：对于多方面任务，使用单一且复杂的提示信息来指导逻辑学习模型（LLM）可能效率低下，导致模型难以应对各种约束和指令，进而可能出现指令忽略（即忽略提示信息的部分内容）、上下文漂移（即模型丢失初始上下文）、错误传播（即早期错误不断放大）、需要更长上下文窗口的提示信息（即模型获取的信息不足以做出响应）以及认知负荷过重导致信息错误的可能性增加等问题。例如，要求分析市场调研报告、总结调查结果、识别数据点趋势并撰写电子邮件的查询，就可能出现失败的情况，因为模型或许能够很好地总结结果，但却无法正确提取数据或撰写电子邮件。

通过顺序分解提高可靠性：提示链式方法通过将复杂任务分解为专注的、顺序的工作流程来应对这些挑战，从而显著提高可靠性和可控性。以上述示例为例，流水线或链式方法可以描述如下：

1. 初始提示（总结）：“请总结以下市场调研报告的主要发现：[文本]。” 该模型的唯一重点是总结，从而提高初始步骤的准确性。2. 第二个提示（趋势识别）：“根据总结结果，找出前三项趋势。”

找出新兴趋势，并提取支持每项趋势的具体数据点：[步骤 1 的输出]。此提示现在更加具体，并且直接基于已验证的输出。3. 第三个提示（电子邮件撰写）：“为市场营销团队撰写一封简洁的电子邮件，

概述了以下趋势及其支持数据：[步骤 2 的输出]。

这种分解方式能够对流程进行更精细的控制。每个步骤都更加简单明了，从而降低了模型的认知负荷，并最终产生更准确可靠的输出。这种模块化类似于计算流水线，其中每个函数执行特定的操作，然后将结果传递给下一个函数。为了确保对每个特定任务都能做出准确的响应，可以在每个阶段为模型分配不同的角色。例如，在给定的场景中，初始提示可以指定为“市场分析师”，后续提示可以指定为“交易分析师”，第三个提示可以指定为“专家文档撰写员”，依此类推。

结构化输出的作用：提示链的可靠性高度依赖于各步骤之间传递的数据的完整性。如果一个提示的输出含糊不清或格式错误，后续提示可能会因为输入错误而失败。为了避免这种情况，指定结构化的输出格式（例如 JSON 或 XML）至关重要。

例如，趋势识别步骤的输出可以格式化为 JSON 对象：



```
{"趋势": [
    {
"trend_name": "AI驱动的个性化", "supporting_data": "73%的消费者更喜欢与
那些利用个人信息来提升购物体验相关性的品牌进行交易。"
    }
    ,
    {
"trend_name": "可持续和道德品牌", "supporting_data": "过去五年，带有
ESG 相关声明的产品的销售额增长了 28%，而没有此类声明的产品的销售额仅增长了
20%"
    } ]}
```

这种结构化格式确保数据可被机器读取并能被精确解析。并毫无歧义地插入到下一个提示中。这种做法最大限度地减少了自然语言解释中可能出现的错误，是构建稳健的多步骤LLM系统的关键组成部分。

# 实际应用及案例

提示链是一种用途广泛的模式，适用于构建智能体系统的各种场景。它的核心优势在于将复杂问题分解为一系列顺序的、可管理的步骤。以下是一些实际应用和用例：

**1.** 信息处理工作流程：许多任务涉及对原始信息进行多次转换。例如，对文档进行摘要，提取关键实体，然后使用这些实体查询数据库或生成报告。一个典型的操作流程可能如下所示：

- 提示 1：从给定的 URL 或文档中提取文本内容。
- 提示 2：总结清理后的文本。
- 提示 3：从摘要或原文中提取特定实体（例如，名称、日期、地点）。
- 提示 4：使用这些实体搜索内部知识库。
- 提示 5：生成包含摘要、实体和搜索结果的最终报告。

该方法应用于自动化内容分析、人工智能驱动的研究助手开发以及复杂报告生成等领域。



**2.** 复杂查询应答：回答需要多步骤推理或信息检索的复杂问题是其主要应用场景。例如，“1929 年股市崩盘的主要原因是什么？政府政策是如何应对的？”

- 提示 1：确定用户查询中的核心子问题（事故原因、政府应对措施）。
- 提示 2：研究或检索有关 1929 年空难原因的具体信息。
- 提示 3：研究或检索有关政府对 1929 年股市崩盘的政策应对措施的具体信息。
- 提示 4：将步骤 2 和 3 中的信息综合起来，形成对原始问题的连贯答案。

这种顺序处理方法是开发能够进行多步骤推理和信息综合的人工智能系统的关键所在。当一个查询无法仅凭单个数据点回答，而需要一系列逻辑步骤或整合来自不同来源的信息时，就需要这类系统。

例如，一个旨在生成特定主题综合报告的自动化研究代理会执行混合计算工作流程。首先，系统会检索大量相关文章。随后，可以从每篇文章中提取关键信息，这项任务可以针对每个来源并行执行。这一阶段非常适合并行处理，通过同时运行独立的子任务来最大限度地提高效率。

然而，一旦各个数据提取步骤完成，整个流程就呈现出固有的顺序性。系统必须首先整理提取的数据，然后将其综合成一份连贯的草稿，最后审核并完善这份草稿，最终生成一份报告。每个后续阶段在逻辑上都依赖于前一个阶段的成功完成。这就是提示链的应用之处

:整理后的数据作为综合提示的输入，而生成的综合文本则作为最终审核提示的输入。因此，复杂的操作通常会结合并行处理（用于独立数据收集）和提示链（用于综合和完善这两个依赖步骤）。

**3.** 数据提取和转换：将非结构化文本转换为结构化格式通常是通过迭代过程实现的，需要进行顺序修改以提高输出的准确性和完整性。

- 提示 1：尝试从发票文档中提取特定字段（例如，姓名、地址、金额）。



- 处理：检查是否已提取所有必需字段，以及是否符合格式要求。
- 提示 2（条件）：如果字段缺失或格式错误，则创建一个新的提示，要求模型专门查找缺失/格式错误的信息，并可能提供失败尝试的上下文。
- 后续步骤：再次验证结果。如有必要，重复上述步骤。
- 输出：提供提取的、经过验证的结构化数据。

这种顺序处理方法尤其适用于从非结构化数据源（例如表单、发票或电子邮件）中提取和分析数据。例如，解决复杂的光学字符识别 (OCR) 问题（例如处理 PDF 表单）通过分解的多步骤方法可以更有效地完成。

首先，系统采用大型语言模型（LLM）从文档图像中提取文本。随后，模型处理原始输出以规范化数据，例如将“一千零五十”等数值文本转换为其等效值 1050。LLM 面临的一大挑战是执行精确的数学运算。因此，在后续步骤中，系统可以将所需的算术运算委托给外部计算器工具。LLM 识别必要的计算，将规范化后的数字输入到工具中，然后应用计算结果。这种文本提取、数据规范化和使用外部工具的链式流程最终能够获得精确的结果，而这通常难以通过单个 LLM 查询可靠地获得。

**4.** 内容生成工作流程：复杂内容的创作是一个程序性任务，通常分解为不同的阶段，包括初步构思、结构概述、撰写初稿和后续修改。

- 提示 1：根据用户的普遍兴趣生成 5 个主题想法。
- 处理方式：允许用户选择一个想法或自动选择最佳想法。
- 提示 2：根据所选主题，生成详细大纲。
- 提示 3：根据提纲中的第一点，写一个草稿章节。
- 提示 4：根据提纲中的第二点，撰写一个草稿章节，并提供前一章节作为背景。对所有提纲要点重复此操作。

- 提示 5:审阅并完善完整草稿，使其连贯、语气和语法正确。

该方法适用于一系列自然语言生成任务，包括自动生成创意叙事、技术文档和其他形式的结构化文本内容。

**5.** 具有状态的对话代理:尽管全面的状态管理架构采用的方法比顺序链接更复杂，但提示链接为保持对话连续性提供了一种基础机制。这种技术维持了



通过将每个对话回合构建成一个新的提示，系统地将对话序列中先前互动的信息或提取的实体纳入其中，从而构建上下文。

- 提示 1:处理用户话语 1，识别意图和关键实体。
- 处理:使用意图和实体更新对话状态。
- 提示 2:根据当前状态，生成响应和/或确定下一个所需信息。
- 对后续回合重复上述步骤，每个新用户的发言都会启动一个利用不断积累的对话历史（状态）的链。

这一原则对于对话代理的开发至关重要，它使代理能够在长时间的多轮对话中保持上下文的连贯性和一致性。通过保留对话历史，系统能够理解并恰当地响应依赖于先前交流信息的用户输入。

**6.** 代码生成与优化:功能代码的生成通常是一个多阶段过程，需要将问题分解成一系列离散的逻辑操作，然后逐步执行这些操作。

- 提示 1:理解用户对代码功能的请求。生成伪代码或概要。
- 提示 2:根据大纲编写初始代码草稿。
- 提示 3:识别代码中潜在的错误或需要改进的地方（或许可以使用静态分析工具或其他 LLM 调用）。
- 提示 4:根据已发现的问题重写或改进代码。
- 提示 5:添加文档或测试用例。

在人工智能辅助软件开发等应用中，提示链的实用性源于其将复杂的编码任务分解为一系列易于管理的子问题的能力。这种模块化结构降低了大型语言模型在每个步骤中的操作复杂度。至关重要的是，这种方法还允许在模型调用之间插入确定性逻辑，从而在工作流中实现中间数据处理、输出验证和条件分支。通过这种方法，原本可能导致不可靠或不完整结果的单个多方面请求被转

换为由底层执行框架管理的结构化操作序列。

**7.** 多模态和多步骤推理：分析包含多种模态的数据集需要将问题分解成更小的、基于提示的任务。例如，解读一张包含嵌入文本的图片、突出显示特定文本片段的标签以及解释每个标签的表格数据的图像，就需要采用这种方法。



- 提示 1：从用户的图像请求中提取并理解文本。
- 提示 2：将提取的图像文本与其对应的标签关联起来。
- 提示 3：使用表格解释收集到的信息，以确定所需的输出。

# 动手实践代码示例

实现提示链的方法多种多样，从脚本中直接按顺序调用函数，到利用专门设计的框架来管理控制流、状态和组件集成，不一而足。诸如 LangChain、LangGraph、Crew AI 和 Google Agent Development Kit (ADK) 等框架为构建和执行这些多步骤流程提供了结构化的环境，这对于复杂的架构尤为有利。

为了演示，LangChain 和 LangGraph 是合适的选择，因为它们的核心 API 专门设计用于构建操作链和操作图。LangChain 为线性序列提供基础抽象，而 LangGraph 则扩展了这些功能，以支持有状态和循环计算，这对于实现更复杂的智能体行为至关重要。本示例将重点介绍一个基本的线性序列。

以下代码实现了一个两步提示链，作为数据处理管道。初始阶段旨在解析非结构化文本并提取特定信息。后续阶段接收提取的输出并将其转换为结构化数据格式。

要复现此过程，必须先安装所需的库。可以使用以下命令完成此操作：

```
pip install langchain langchain-community langchain-openai
langgraph
```

请注意，langchain-openai 可以替换为其他模型提供商的相应软件包。随后，必须使用所选语言模型提供商（例如 OpenAI、Google Gemini 或 Anthropic）所需的 API 凭据配置执行环境。

```
从 langchain_openai 导入 ChatOpenAI，从
langchain_core.prompts 导入 ChatPromptTemplate，从
```

```
langchain_core.output_parsers 导入 StrOutputParser

# 为了提高安全性，请从 .env 文件加载环境变量 # from dotenv import
load_dotenv
```



```
# load_dotenv() # 请确保已在 .env 文件中设置
OPENAI_API_KEY

# 初始化语言模型（推荐使用 ChatOpenAI） llm =
ChatOpenAI(temperature=0)

# --- 提示 1：提取信息 --- prompt_extract =
ChatPromptTemplate.from_template(

  请从以下文本中提取技术规格：\n\n{text_input})# --- 提示 2：转

换为 JSON ---prompt_transform =
ChatPromptTemplate.from_template(
  “将以下规格转换为以 'cpu'、'memory' 和 'storage' 为键的 JSON 对象：

\n\n{specifications}”# --- 使用LCEL构建链 ---# StrOutputParser() 将
LLM 的消息输出转换为简单的字符串。extraction_chain = prompt_extract |
llm | StrOutputParser()

# 完整链将提取链的输出传递给转换提示的“specifications”变量。full_chain =
(
{"specifications": extraction_chain} | prompt_transform | llm

| StrOutputParser() )# --- 运行链式调用 ---input_text = "这款新
型笔记本电脑配备了 3.5 GHz 八核处理器、16GB 内存和 1TB NVMe 固态硬
盘。"

# 使用输入文本字典执行链式调用。final_result =
full_chain.invoke({"text_input": input_text})

print("\n--- 最终 JSON 输出 ---")
print(final_result)
```

这段 Python 代码演示了如何使用 LangChain 库处理文本。它使用了两个独立的提示：一个用于从输入字符串中提取技术规范，另一个用于将这些规范格式化为 JSON 对象。ChatOpenAI 模

型用于语言模型交互，而 StrOutputParser 则确保输出为可用的字符串。



格式方面，我们使用 LangChain 表达式语言 (LCEL) 将这些提示和语言模型优雅地串联起来。第一个链 extraction_chain 用于提取规范。full_chain 则将提取的输出作为转换提示的输入。我们提供了一个描述笔记本电脑的示例输入文本。full_chain 会使用该文本，并依次经过两个步骤进行处理。最终结果是一个包含已提取和格式化规范的 JSON 字符串，并会被打印出来。

# 上下文工程和提示工程

上下文工程（见图1）是一门系统性的学科，它旨在人工智能模型生成令牌之前，为其设计、构建并提供完整的上下文信息环境。该方法论认为，模型输出的质量与其说是取决于模型架构本身，不如说是取决于所提供上下文的丰富程度。

图 1：上下文工程是为人工智能构建丰富、全面的信息环境的学科，因为上下文的质量是实现高级智能体性能的主要因素。

它代表着对传统提示工程的重大革新，传统提示工程主要关注优化用户直接查询的措辞。而上下文工程则扩展了这一范围，纳入了多层信息，例如：系统提示这是一套定义人工智能运行的基本指令集。

参数——例如，“你是技术写作人员；你的语气必须正式、准确。”上下文信息会进一步通过外部数据得到丰富。这包括检索到的文档，人工智能会主动从知识库中获取信息以指导其响应，例如提取项目的技术规范。它还整合了工具输出，这些输出是人工智能使用外部API获取实时数据的结果，例如查询日历以确定用户的可用时间。这些显式数据与关键的隐式数据相结合，例如用户身份、交互历史和环境状态。其核心原则是，即使是高级模型，如果对运行环境的了解有限或构建不完善，其性能也会下降。

因此，这种做法将任务从单纯回答问题转变为为智能体构建全面的操作图景。例如，经过情境工程设计的智能体不仅会响应查询，还会首先整合用户的日程安排（工具输出）、与邮件收件人的专业关系（隐式数据）以及以往会议的记录（检索文档）。这使得模型能够生成高度相关、个性化且实用的输出。“工程”部分涉及创建强大的管道，以便在运行时获取和转换这些数据，并建立反馈循环以持续改进情境质量。

为了实现这一点，可以使用专门的调优系统来大规模地自动化改进过程。例如，像谷歌的 Vertex AI 提示优化器这样的工具，可以通过系统地评估模型响应与一组示例输入和预定义的评估指标之间的关系来提升模型性能。这种方法能够有效地在不同模型之间调整提示和系统指令，而无需进行大量的手动重写。通过向此类优化器提供示例提示、系统指令和模板，它可以以编程方式优化上下文输入，从而为实现复杂上下文工程所需的反馈循环提供一种结构化的方法。

这种结构化的方法正是区分初级人工智能工具和更复杂、更具情境感知能力的系统的关键所在。它将情境本身视为核心组成部分，高度重视智能体掌握的信息、获取信息的时间以及如何运用这些信息。这种做法确保模型能够全面理解用户的意图、历史记录和当前环境。最终，情境工程是推动无状态聊天机器人发展成为功能强大、具备情境感知能力的系统的关键方法。

# 概览

什么：当复杂任务在单个提示中处理时，往往会使逻辑学习模型（LLM）不堪重负，导致严重的性能问题。模型的认知负荷增加。



诸如忽略指令、丢失上下文以及生成错误信息等错误的可能性很高。单一的提示难以有效管理多个约束条件和顺序推理步骤。这会导致输出不可靠且不准确，因为逻辑逻辑模型

(LLM) 无法处理多方面请求的所有细节。

为什么：提示链通过将复杂问题分解为一系列相互关联的子任务，提供了一种标准化的解决方案。链中的每个步骤都使用一个聚焦的提示来执行特定操作，从而显著提高可靠性和可控性。一个提示的输出作为下一个提示的输入，形成一个逻辑工作流，逐步构建最终解决方案。这种模块化的分而治之策略使流程更易于管理、调试，并允许在不同步骤之间集成外部工具或结构化数据格式。这种模式是开发能够规划、推理和执行复杂工作流的复杂多步骤智能体系统的基础。

经验法则：当任务过于复杂，无法通过单个提示完成，涉及多个不同的处理阶段，需要在步骤之间与外部工具进行交互，或者构建需要执行多步骤推理和维护状态的智能体系统时，可以使用此模式。

可视化摘要

图 2：提示链模式：代理接收来自用户的一系列提示，每个代理的输出作为链中下一个代理的输入。

# 要点总结

以下是一些关键要点：

- 提示链将复杂任务分解成一系列更小、更专注的步骤。这有时也被称为流水线模式。
- 链中的每一步都涉及 LLM 调用或处理逻辑，使用上一步的输出作为输入。
- 这种模式提高了与语言模型进行复杂交互的可靠性和可管理性。
- LangChain/LangGraph 和 Google ADK 等框架提供了强大的工具来定义、管理和执行这些多步骤序列。

# 结论

通过将复杂问题分解为一系列更简单、更易于管理的子任务，提示链为指导大型语言模型提供了一个强大的框架。这种“分而治之”的策略通过让模型一次专注于一个特定的操作，显著提高了输出的可靠性和可控性。作为一种基础模式，它能够开发出具备多步骤推理、工具集成和状态管理能力的复杂人工智能代理。最终，掌握提示链对于构建能够执行远超单个提示所能处理的复杂工作流程的强大、上下文感知系统至关重要。

# 参考

1. LangChain 关于 LCEL 的文档：https://python.langchain.com/v0.2/docs/core_modules/expression_language/2. LangGraph 文档：https://langchain-ai.github.io/langgraph/3. 提示工程指南 - 链接提示：https://www.promptingguide.ai/techniques/chaining4. OpenAI API 文档（通用提示概念）：https://platform.openai.com/docs/guides/gpt/prompting5. 机组人员人工智能文档（任务和流程）：https://docs.crewai.com/6. Google AI for Developers（提示指南）：https://cloud.google.com/discover/what-is-prompt-engineering?hl=en7. 顶点提示优化器 https://cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/prompt-optimizer

# 第二章：路由

## 路由模式概述

虽然通过提示链进行顺序处理是使用语言模型执行确定性线性工作流的基础技术，但其在需要自适应响应的场景中的适用性有限。现实世界中的智能体系统通常必须根据环境状态、用户输入或先前操作的结果等偶然因素，在多个潜在动作之间进行权衡。这种动态决策能力控制着流向不同专用功能、工具或子流程的进程，而路由机制正是实现这一能力的关键。

路由机制将条件逻辑引入到智能体的操作框架中，使其执行路径从固定不变转变为动态评估特定标准，并从一系列可能的后续动作中进行选择。这使得系统行为更加灵活，并能更好地感知上下文。

例如，专为客户咨询设计的代理，如果配备了路由功能，可以首先对传入的查询进行分类，以确定用户的意图。基于此分类，它可以将查询定向到专门的客服人员进行直接问答、数据库检索工具用于查找账户信息，或用于处理复杂问题的升级流程，而不是默认使用单一的预设响应路径。因此，使用路由功能的更高级代理可以：

1.分析用户查询。2.路线 基于其的查询意

图：

○如果意图是“检查订单状态”，则路由到与订单数据库交互的子代理或工具链。

○如果目的是“产品信息”，则路由至搜索产品目录的子代理商或连锁店。

○如果目的是“技术支持”，则转接到可以访问故障排除指南或升级到人工支持的其他流程。

○如果意图不明确，则转接至澄清子代理或提示链。

路由模式的核心组件是执行评估并指导流程的机制。该机制可以通过多种方式实现：

●基于**LLM**的路由：语言模型本身可以被提示分析输入，并输出一个特定的标识符或指令，以指示下一步或目标位置。例如，提示信息可以要求语言模型“分析以下内容”。

1

用户查询并仅输出以下类别：“订单状态”、“产品信息”、“技术支持”或“其他”。然后，代理系统读取此输出并据此指导工作流程。

●基于嵌入式的路由：输入查询可以转换为向量嵌入（参见 RAG，第 14 章）。然后，将此嵌入与表示不同路径或功能的嵌入进行比较。查询将被路由到嵌入最相似的路径。这对于语义路由非常有用，因为语义路由的决策基于输入的含义，而不仅仅是关键词。

●基于规则的路由：这涉及到使用基于从输入中提取的关键字、模式或结构化数据的预定义规则或逻辑（例如，if-else 语句、switch case）。与基于 LLM 的路由相比，这种方法速度更快、确定性更高，但在处理细微差别或新颖的输入时灵活性较差。

●基于机器学习模型的路由它采用判别模型（例如分类器），该模型已在一小批带标签的数据集上进行专门训练，用于执行路由任务。虽然它在概念上与基于嵌入的方法有相似之处，但其关键特征在于监督式微调过程，该过程调整模型参数以创建专门的路由功能。该技术与基于LLM的路由不同，因为其决策组件并非在推理时执行提示的生成模型。相反，路由逻辑编码在微调模型学习到的权重中。虽然LLM可用于预处理步骤以生成合成数据来扩充训练集，但它们本身并不参与实时路由决策。

路由机制可以在代理的运行周期中的多个节点实施。它们可以在开始时应用以对主要任务进行分类，在处理链中的中间点应用以确定后续操作，或者在子程序期间应用以从给定的集合中选择最合适的工具。

诸如 LangChain、LangGraph 和 Google 的 Agent Developer Kit (ADK) 等计算框架提供了用于定义和管理此类条件逻辑的显式结构。凭借其基于状态的图架构，LangGraph 特别适用于复杂的路由场景，在这些场景中，决策取决于整个系统的累积状态。类似地，Google 的 ADK 提供了用于构建代理能力和交互模型的基础组件，这些组件构成了实现路由逻辑的基础。在这些框架提供的执行环境中，开发人员可以定义可能的操作路径，并根据这些路径进行路由。

决定计算图中节点间转换的函数或基于模型的评估。

路由机制的实施使系统能够超越确定性的顺序处理。它有助于开发更具适应性的执行流程，从而能够动态且适当地响应更广泛的输入和状态变化。

# 实际应用及案例

路由模式是自适应智能体系统设计中的关键控制机制，它使系统能够根据变化的输入和内部状态动态地改变执行路径。路由模式通过提供必要的条件逻辑层，在多个领域都具有实用价值。

在人机交互中，例如与虚拟助手或人工智能驱动的辅导系统交互时，路由机制用于解读用户意图。对自然语言查询的初步分析会确定最合适的后续操作，例如调用特定的信息检索工具、转接人工客服，或根据用户表现选择课程中的下一个模块。这使得系统能够超越线性对话流程，并根据上下文做出响应。

在自动化数据和文档处理流程中，路由扮演着分类和分发的角色。系统会根据内容、元数据或格式分析传入的数据，例如电子邮件、支持工单或 API 请求。然后，系统会将每个数据项定向到相应的工作流程，例如销售线索导入流程、针对 JSON 或 CSV 格式的特定数据转换功能，或紧急问题升级路径。

在涉及多个专用工具或代理的复杂系统中，路由机制扮演着高级调度器的角色。例如，一个由用于搜索、汇总和分析信息的不同代理组成的科研系统，会利用路由机制，根据当前目标将任务分配给最合适的代理。同样，人工智能编码助手也会利用路由机制来识别编程语言和用户的意图（例如调试、解释或翻译），然后再将代码片段传递给正确的专用工具。

最终，路由提供了逻辑仲裁的能力，这对于创建功能多样化且具有上下文感知能力的系统至关重要。它将代理从预定义序列的静态执行器转变为能够进行动态交互的系统。



在不断变化的情况下，如何选择完成任务的最有效方法做出决策。

# 动手实践代码示例（LangChain）

在代码中实现路由涉及定义所有可能的路径以及决定走哪条路径的逻辑。LangChain 和 LangGraph 等框架为此提供了特定的组件和结构。LangGraph 基于状态的图结构尤其便于可视化和实现路由逻辑。

这段代码演示了一个使用 LangChain 和 Google 生成式人工智能的简单类智能体系统。它设置了一个“协调器”，根据用户请求的意图（预订、信息或不明确）将请求路由到不同的模拟“子智能体”处理程序。该系统使用语言模型对请求进行分类，然后将其委托给相应的处理函数，模拟了多智能体架构中常见的基本委托模式。

首先，请确保已安装必要的库：

```
pip install langchain langgraph
google-cloud-aiplatform langchain-google-genai
google-adk deprecated pydantic
```

您还需要使用您选择的语言模型（例如 OpenAI、Google Gemini、Anthropic）的 API 密钥来设置您的环境。

```
# 版权所有 (c) 2025 Marco Fago #
https://www.linkedin.com/in/marco-fago/ ## 本代码采用 MIT 许可证授
权。# 有关完整的许可文本，请参阅存储库中的 LICENSE 文件。

from langchain_google_genai import
ChatGoogleGenerativeAI from langchain_core.prompts
import ChatPromptTemplate from
langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import
RunnablePassthrough, RunnableBranch

# --- 配置 --- # 请确保已设置 API 密钥环境变量（例如，
GOOGLE_API_KEY） try:llm =
ChatGoogleGenerativeAI(model="gemini-2.5-flash",
```



```
温度=0)
```

print(f"语言模型已初始化：{llm.model}") except Exception as e:
print(f"初始化语言模型时出错：{e}") llm = None

# --- 定义模拟子代理处理器（等同于 ADK 子代理） ---

def booking_handler(request: str) -> str:
"""模拟预订代理处理请求。""" print("\n--- 委托给预订处理程序 ---") return f"预订处理程序已处理请求：'{request}'。结果：模拟预订操作。"""

def info_handler(request: str) -> str:
"""模拟信息代理处理请求。""" print("\n--- 委托给信息处理程序 ---") return f"信息处理程序已处理请求：'{request}'。结果：模拟信息检索。"""

def unknown_handler(request: str) -> str:
处理无法委派的请求。 print("\n--- 处理不明确的请求 ---") return f"协调器无法委派请求：'{request}'。请澄清。"

# --- 定义协调器路由链（等效于 ADK 协调器的指令）--- # 此链决定委托给哪个处理程序。 coordinator_router_prompt = ChatPromptTemplate.from_messages([
("系统", """分析用户的请求，并确定哪个专家处理程序应该处理它。
- 如果请求与预订机票或酒店有关，
输出"预订者"。- 对于所有其他一般信息问题，输出"信息"。- 如果请求不清楚或不属于上述任何一类，
输出"unclear"。仅输出一个单词："booker"、"info"或"unclear"。
"""), ("user", "{request}") ])如果 llm：
coordinator_router_chain = coordinator_router_prompt | llm | StrOutputParser()

# --- 定义委托逻辑（等效于 ADK 的自动流）



基于子代理）--- # 使用 RunnableBranch 根据路由器链的输出进行路由。

# 定义 RunnableBranch 的分支 branches = {

"booker": RunnablePassthrough.assign(output=lambda x: booking_handler(x['request']['request'])),

"info": RunnablePassthrough.assign(output=lambda x: info_handler(x['request']['request'])),

"unclear": RunnablePassthrough.assign(output=lambda x: unknown_handler(x['request']['request'])), }# 创建可运行分支。它接收路由链的输出。# 并将原始输入（"请求"）路由到相应的处理程序。 delegate_branch = RunnableBranch(

(lambda x: x['decision'].strip() == 'booker', branches["booker"]), # 添加了 .strip()

(lambda x: x['decision'].strip() == 'info', branches["info"]), # 添加了 .strip()

branches["unclear"] # 'unclear' 或任何其他输出的默认分支 )# 将路由链和委托分支合并成一个分支runnable # 路由链的输出（"决策"）与原始输入（"请求"）一起传递给 delegate_branch。 coordinator_agent = { "decision": coordinator_router_chain, "request": RunnablePassthrough() } | delegate_branch | (lambda x: x['output']) # 提取最终输出

# --- 用法示例 --- def main():

如果不是llm:

print("\n由于LLM初始化失败，跳过执行。")

返回

print("--- 正在处理预订请求 ---") request_a = "请帮我预订飞往伦敦的航班。" result_a = coordinator_agent.invoke({"request": request_a}) print(f"最终结果 A：{result_a}")



print("\n--- 正在运行信息请求 ---") request_b = "意大利的首都是什么？" result_b = coordinator_agent.invoke({"request": request_b}) print(f"最终结果 B：{result_b}")

print("\n--- 正在处理不明确的请求 ---") request_c = "请告诉我

```
    一些关于量子物理的知识。" result_c =
    coordinator_agent.invoke({"request": request_c}) print(f"
    最终结果 C：{result_c}")

如果 __name__ ==
"__main__":
    主要的
    ()
```

如前所述，这段 Python 代码使用 LangChain 库和 Google 的生成式 AI 模型（具体来说是 gemini-2.5-flash）构建了一个简单的类代理系统。具体来说，它定义了三个模拟子代理处理器：booking_handler、info_handler 和 unknown_handler，每个处理器都用于处理特定类型的请求。

核心组件是 coordinator_router_chain，它利用 ChatPromptTemplate 指示语言模型将传入的用户请求分类为三个类别之一："预订"、"信息"或"不清楚"。然后，RunnableBranch 使用此路由链的输出将原始请求委托给相应的处理函数。RunnableBranch 检查语言模型的决策，并将请求数据定向到 booking_handler、info_handler 或 unknown_handler。coordinator_agent 将这些组件组合起来，首先路由请求以进行决策，然后将请求传递给选定的处理函数。最终输出从处理函数的响应中提取。

主函数通过三个示例请求演示了系统的使用方法，展示了模拟代理如何路由和处理不同的输入。为了确保系统的健壮性，代码中包含了语言模型初始化的错误处理机制。代码结构模拟了一个基本的多代理框架，其中中央协调器根据意图将任务委派给专门的代理。

# 动手实践代码示例（Google ADK）

智能体开发工具包（ADK）是一个用于构建智能体系统的框架，它提供了一个结构化的环境来定义智能体的能力和行为。与基于显式计算图的架构相比，



在 ADK 框架中，路由通常是通过定义一组离散的"工具"来实现的，这些工具代表了代理的功能。框架的内部逻辑会根据用户查询选择合适的工具，并利用底层模型将用户意图与正确的功能处理程序进行匹配。

这段 Python 代码演示了一个使用 Google ADK 库的代理开发工具包 (ADK) 应用程序示例。它设置了一个名为"Coordinator"的代理，该代理根据预定义的指令将用户请求路由到

专门的子代理(“Booker”用于预订，“Info”用于提供一般信息)。然后，子代理使用特定的工具来模拟处理请求，从而展示了代理系统中的基本委托模式。

```
版权所有 © 2025 Marco Fago# 本代码采用 MIT 许可证授权。# 有关完整的许可文
本，请参阅存储库中的 LICENSE 文件。

import uuid from typing import Dict,
Any, Optional

from google.adk.agents import Agent from
google.adk.runners import InMemoryRunner
from google.adk.tools import FunctionTool
from google.genai import types from
google.adk.events import Event

# --- 定义工具功能 --- # 这些功能模拟专家代理的操作。

def booking_handler(request: str) ->
str:

    处理航班和酒店的预订请求。参数：请求：用户提出的预订请

    求。返回：
    确认预订已处理的消息。"""
print("-------------------------- 预订处理程序已调用
----------------------------")
    返回 f"已模拟对'{request}'的预订操作。"

def info_handler(request: str) ->
str:
    处理一般信息请求。参数：
```



```
    请求：用户的问题。返回值：
    一条消息，表明信息请求已处理。"""
print("-------------------------- 信息处理程序已调用
----------------------------")
   返回 f"对"{request}"的信息请求。结果：模拟信息检索。"

def unknown_handler(request: str) ->
```

str:
处理无法委托的请求。返回 f"协调器无法委托请求：'{request}'。请澄清。"

# --- 从函数创建工具 --- booking_tool =
FunctionTool(booking_handler) info_tool =
FunctionTool(info_handler)

# 定义配备各自工具的专业子代理 booking_agent = Agent(
名称="Booker", 型号="gemini-2.0-flash", 描述="一款专门处理
所有航班的代理程序"
并通过调用预订工具来提交酒店预订请求。"（tools=[booking_tool]）

info_agent = Agent(名称="信息", 型号="gemini-2.0-flash", 描述="提供一般
信息的专业代理

并通过调用信息工具来回答用户问题。"（tools=[info_tool]）# 使用显式

委托指令定义父代理协调器 = 代理(
名称="协调器", 型号
="gemini-2.0-flash", 指令
=(
你是主要协调员。你的唯一任务是分析
传入的用户请求" "并将它们委派给相应的专业代理。
请勿直接回复用户。\n""- 对于任何与预订航班或酒店相关的请求,
委托给"预订员"代理人。\n""- 对于所有其他一般信息问题，请委托给



"信息"代理。

），描述="将用户请求路由到…的协调器
正确的专家代理。"，# 子代理的存在默认启用 LLM 驱动的委托（自动流）。

子代理=[预订代理，信息代理]）# --- 执行逻辑

---

异步

def run_coordinator(runner: InMemoryRunner, request:
str): """使用给定的请求和委托运行协调器代理。"""
print(f"\n--- 正在运行协调器，请求为：'{request}' ---")

final_result = "" try:用户 ID = "user_123"

session_id = str(uuid.uuid4()) await runner.session_service.create_session (

app_name=runner.app_name, user_id=user_id, session_id=session_id

) 对于 runner.run 中的事件

(user_id=user_id,

session_id=session_id,

new_message=types.Content(

role='user',

parts=[types.Part(text=request)] ), ):

如果 event.is_final_response() 且 event.content:

# 尝试直接从 event.content 获取文本 # 以避免迭代部分 if hasattr(event.content, 'text') and event.content.text:

final_result = event.content.text elif event.content.parts:

# 备选方案：遍历各个部分并提取文本（可能会触发警告）

text_parts = [part.text for part in event.content.parts if part.text]

final_result = "".join(text_parts) # 假设循环在最终结果之后中断

10

回复

休息

print(f"协调员最终响应：{final_result}") return final_result except Exception as e:

print(f"处理您的请求时发生错误：{e}") return f"处理您的请求时发生错误：{e}"

异步

def main():

"""运行 ADK 示例的主函数。"" print("--- Google ADK 路由示例（ADK 自动流样式）---") print("注意：这需要安装并验证 Google ADK。")

```
runner = InMemoryRunner(coordinator) # 示例用法 result_a =
await run_coordinator(runner, "帮我预订巴黎的酒店。")
print(f"最终输出 A：{result_a}") result_b = await
run_coordinator(runner, "世界上最高的山是什么？")
print(f"最终输出 B：{result_b}") result_c = await
run_coordinator(runner, "告诉我一个随机事实。") # 应该输出到 Info
print(f"最终输出 C: {result_c}") result_d = await
run_coordinator(runner, "查找下个月飞往东京的航班。") # 应该使用
Booker
   print(f"最终输出 D: {result_d}")

如果 __name__ ==
"__main__":
   import nest_asyncio
   nest_asyncio.apply(
   ) await main()
```

该脚本包含一个主协调器代理和两个专门的子代理：预订器和信息代理。每个专门代理都配备了一个 FunctionTool，该工具封装了一个模拟操作的 Python 函数。预订处理程序函数模拟处理航班和酒店预订，而信息处理程序函数模拟检索一般信息。虽然当前协调器逻辑在主运行协调器函数中并未显式地在委托失败时使用它，但仍然包含了一个不明确的处理程序作为协调器无法委托请求的备用方案。

1
1

根据其指令定义，协调器代理的主要职责是分析传入的用户消息，并将其委托给 Booker 代理或 Info 代理。由于协调器代理定义了子代理，因此这种委托由 ADK 的自动流机制自动处理。`run_coordinator` 函数会设置一个内存运行器 (InMemoryRunner)，创建用户 ID 和会话 ID，然后使用该运行器通过协调器代理处理用户的请求。`runner.run` 方法处理请求并生成事件，代码会从 `event.content` 中提取最终响应文本。

主要功能通过运行协调器并执行不同的请求来演示系统的使用方法，展示了它如何将预订请求委派给预订员，将信息请求委派给信息代理。

# 概览

什么智能体系统通常需要响应各种各样的输入和情况，而这些无法通过单一的线性流程来处理。简单的顺序工作流缺乏基于上下文进行决策的能力。如果没有机制来为特定任务选择正确的工具或子流程，系统就会变得僵化且缺乏适应性。这种局限性使得构建能够应对现实世界用户请求的复杂性和多变性的复杂应用程序变得困难。

为什么：路由模式通过在代理的操作框架中引入条件逻辑，提供了一种标准化的解决方案。它使系统能够首先分析传入的查询，以确定其意图或性质。基于此分析，代理可以动态地将控制流导向最合适的专用工具、功能或子代理。此决策可以通过多种方法驱动，包括提示逻辑逻辑模型（LLM）、应用预定义规则或使用基于嵌入的语义相似性。最终，路由将静态的、预定的执行路径转换为灵活且上下文感知的工作流程，从而能够选择最佳操作。

经验法则：当代理需要根据用户输入或当前状态在多个不同的工作流程、工具或子代理之间做出选择时，应使用路由模式。对于需要对传入请求进行分类或分流以处理不同类型任务的应用程序而言，路由模式至关重要，例如客户支持机器人需要区分销售咨询、技术支持和账户管理问题。

视觉摘要：

图1：使用LLM作为路由器的路由器布局图

# 要点总结

- 路由使代理能够根据条件动态地决定工作流程中的下一步。
- 它允许智能体处理各种输入并调整其行为，超越线性执行。
- 路由逻辑可以使用 LLM、基于规则的系统或嵌入相似性来实现。
- LangGraph 和 Google ADK 等框架提供了结构化的方式来定义和管理代理工作流中的路由，尽管它们的架构方法有所不同。

# 结论

路由模式是构建真正动态且响应迅速的代理系统的关键步骤。通过实现路由，我们超越了简单的线性执行流程，并且



赋予我们的代理智能决策能力，使其能够处理信息、响应用户输入并利用可用工具或子代理。

我们已经了解了路由技术在各个领域的应用，从客户服务聊天机器人到复杂的数据处理流程。分析输入并根据条件引导工作流程的能力，对于创建能够应对现实世界任务固有变化性的智能体至关重要。

使用 LangChain 和 Google ADK 的代码示例展示了两种不同但有效的路由实现方法。LangGraph 基于图的结构提供了一种直观且明确的方式来定义状态和转换，使其成为具有复杂路由逻辑的多步骤工作流的理想选择。另一方面，Google ADK 通常侧重于定义不同的功能（工具），并依赖于框架将用户请求路由到相应工具处理程序的能力，这对于具有明确定义的一组离散操作的代理来说可能更简单。

掌握路由模式对于构建能够智能应对不同场景并根据上下文提供定制化响应或操作的智能体至关重要。它是创建功能全面且稳健的智能体应用程序的关键组成部分。

# 参考

1.LangGraph 文档：https://www.langchain.com/2.Google Agent 开发者工具包文档：https://google.github.io/adk-docs/

# 第三章：并行化

## 并行化模式概述

在前几章中，我们探讨了用于顺序工作流的提示链式机制和用于动态决策以及不同路径之间转换的路由机制。虽然这些模式至关重要，但许多复杂的智能体任务都涉及多个可以执行的子任务。同时地 而不是一个接一个。这就是……并行化模式变得至关重要。

并行化是指同时执行多个组件，例如 LLM 调用、工具使用，甚至是整个子代理（见图 1）。并行执行无需等待一个步骤完成即可开始下一个步骤，它允许独立任务同时运行，从而显著缩短可分解为独立部分的任务的整体执行时间。

考虑一个旨在研究某个主题并总结研究结果的智能体。一种循序渐进的方法可能是：

1.查找来源 A。2.总结资料A。3.查找来源 B。4.总结资料B。5.综合摘要 A 和 B 得出最终答案。

另一种并行方法是：

1.搜索来源 A和 同时搜索源B。2.两项搜索完成后，汇总来源 A。和 总结来源 B 同时地。3.根据摘要 A 和 B 综合得出最终答案（这一步通常是顺序执行，等待并行步骤完成）。

其核心思想是识别工作流程中不依赖于其他部分输出的部分，并并行执行这些部分。这在处理存在延迟的外部服务（例如 API 或数据库 ）时尤为有效，因为您可以同时发出多个请求。

实现并行化通常需要支持异步执行或多线程/多进程的框架。现代代理框架是



其设计充分考虑了异步操作，让您可以轻松定义可以并行运行的步骤。

图1. 子代理并行化示例

LangChain、LangGraph 和 Google ADK 等框架提供了并行执行机制。在 LangChain 表达式语言 (LCEL) 中，您可以通过使用诸如 |（表示顺序）之类的运算符组合可运行对象，以及通过构建链或图的结构使其包含可并发执行的分支来实现并行执行。LangGraph 的图结构允许您定义多个节点，这些节点可以从单个状态转换中执行，从而有效地在工作流中实现并行分支。Google ADK 提供了强大的原生机制来促进和管理代理的并行执行，从而显著提高复杂多代理系统的效率和可扩展性。ADK 框架的这种固有功能使开发人员能够设计和实现多个代理可以并发而非顺序运行的解决方案。



并行化模式对于提升智能体系统的效率和响应速度至关重要，尤其是在处理涉及多次独立查找、计算或与外部服务交互的任务时。它是优化复杂智能体工作流程性能的关键技术。

# 实际应用及案例

并行化是一种强大的模式，可用于优化各种应用程序中的代理性能：

1. 信息收集和研究：同时从多个来源收集信息是一个经典的应用案例。
   - 使用案例：一名正在调查公司的代理人。
     - 并行任务：同时搜索新闻文章、提取股票数据、查看社交媒体提及情况并查询公司数据库。
     - 益处：比顺序查找更快地获取全面信息。
2. 数据处理和分析：同时应用不同的分析技术或处理不同的数据段。
   - 使用案例：一名客服人员正在分析客户反馈。
     - 并行任务：同时对一批反馈条目进行情感分析、提取关键词、对反馈进行分类并识别紧急问题。
     - 益处：快速提供多方面分析。
3. 多 API 或工具交互：调用多个独立的 API 或工具来收集不同类型的信息或执行不同的操作。
   - 使用案例：旅行规划代理人。
     - 并行任务：同时查看航班价格、搜索酒店空房情况、查找当地活动并获取餐厅推荐。
     - 益处：更快地提供完整的旅行计划。
4. 多组件内容生成：并行生成复杂内容的不同部分。
   - 使用案例：一名经纪人正在撰写营销邮件。
     - 并行任务：同时生成邮件主题、撰写邮件正文、查找相关图片并创建行动号召按钮文本。
     - 益处：更高效地生成最终邮件。
5. 验证与确认：



同时执行多个独立的检查或验证。

   - 使用案例：代理正在验证用户输入。
     - 并行任务：同时检查电子邮件格式、验证电话号码、对照数据库验证地址，并检

查是否存在不雅用语。

○益处：能够更快地反馈输入有效性信息。

6. 多模态处理：同时处理同一输入的不同模态（文本、图像、音频）。

●使用案例：一名代理人正在分析包含文字和图片的社交媒体帖子。

○并行任务：分析文本的情感和关键词和 同时分析图像中的物体和场景描述。

○益处：能够更快地整合来自不同模式的信息。

7. A/B 测试或多选项生成：并行生成响应或输出的多个变体，以选择最佳变体。

●使用案例：一个能够生成不同创意文本选项的代理。

○并行任务：使用略有不同的提示或模板，同时为一篇文章生成三个不同的标题。

○益处：便于快速比较和选择最佳方案。

并行化是智能体设计中的一项基本优化技术，它允许开发人员利用独立任务的并发执行来构建性能更高、响应更快的应用程序。

# 动手实践代码示例（LangChain）

LangChain框架内的并行执行由LangChain表达式语言（LCEL）实现。其主要方法是将多个可运行组件组织成字典或列表结构。当此集合作为输入传递给链中的后续组件时，LCEL运行时会并发执行其中包含的可运行组件。

在 LangGraph 的上下文中，这一原则应用于图的拓扑结构。并行工作流通过构建图的结构来实现，使得多个缺乏直接顺序依赖关系的节点可以从同一个公共节点启动。这些并行路径独立执行，之后它们的结果可以在图中的后续汇聚点进行聚合。

以下实现展示了使用 LangChain 框架构建的并行处理工作流。该工作流旨在执行两个任务。4
针对单个用户查询，同时执行多个独立的操作。这些并行进程被实例化为不同的链或函数，它们的各自输出随后被聚合为一个统一的结果。

此实现的前提条件包括安装必要的 Python 包，例如 langchain、langchain-community 以及模型提供程序库（例如 langchain-openai）。此外，还必须在本地环境中配置所选语言模型的有效 API 密钥以进行身份验证。

```
import os import asyncio
```

```
from typing import
Optional

from langchain_openai import ChatOpenAI from
langchain_core.prompts import ChatPromptTemplate from
langchain_core.output_parsers import StrOutputParser from
langchain_core.runnables import Runnable, RunnableParallel,
RunnablePassthrough

# --- 配置 --- # 请确保已设置 API 密钥环境变量（例如，

OPENAI_API_KEY） try:llm: 可选[ChatOpenAI] =

ChatOpenAI(model="gpt-4o-mini",温度=0.7)

除异常 e 外：
   print(f"初始化语言模型时出错：{e}") llm = None

# --- 定义独立链 --- # 这三条链代表可以并行执行的不同任务。

summarize_chain: Runnable =
(
   ChatPromptTemplate.from_messages(
   [

（"系统", "请简要概括以下主题：", "用户", "{主题}"） ])| llm|

StrOutputParser() )questions_chain: Runnable = (
   ChatPromptTemplate.from_messages(
   [
```



```
      （“系统”，“围绕以下主题生成三个有趣的问题：”）

    （"用户", "{主题}"） ])|

llm| StrOutputParser()

)terms_chain: Runnable =

(
   ChatPromptTemplate.from_messages(
   [
```

（“系统”，“从以下主题中找出 5-10 个关键术语，用逗号分隔：”）

("用户", "{主题}") ])| llm|

StrOutputParser() )# --- 构建并行合成链 ---

# 1. 定义要并行运行的任务块。这些任务的结果，连同原始主题，将传递给下一步。
map_chain = RunnableParallel(

{

"summary": summarize_chain, "questions": questions_chain,
"key_terms": terms_chain, "topic": RunnablePassthrough(), # 将原始主题传递 } )2. 定义最终的综合提示，该提示将结合以下内容：并行结果。
synthesis_prompt = ChatPromptTemplate.from_messages([

("系统", """基于以下信息：
摘要：{summary} 相关问题：{questions} 关键词：{key_terms} 综合分析，得出全面答案。""), "user", "原始主题：{topic}") ])3. 通过直接传递并行结果来构建完整的链。在合成提示符中输入 #，然后是 LLM 和输出解析器。



full_parallel_chain = map_chain | synthesis_prompt | llm
| StrOutputParser()

# --- 运行链式调用 --- async def
run_parallel_example(topic: str) -> None:
异步调用具有特定主题的并行处理链

并打印合成结果。

参数：主题：LangChain要处理的输入主题链条。

如果不是 llm

：

print("LLM 未初始化。无法运行示例。") 返回

print(f"\n--- 运行并行 LangChain 示例，主题：'{topic}' ---")

尝试： # `ainvoke` 的输入是单个字符串“topic”，# 然后传递给
`map_chain` 中的每个可运行对象。response = await
full_parallel_chain.ainvoke(topic) print("\n--- 最终响应
---") print(response) except Exception as e:

```
        print(f"\n链式执行过程中发生错误：{e}")

如果 __name__ ==
"__main__":
test_topic = "太空探索的历史" # 在 Python 3.7+ 中，asyncio.run 是运行异
步函数的标准方法。
    asyncio.run(run_parallel_example(test_topic
    ))
```

提供的 Python 代码实现了一个 LangChain 应用，旨在利用并行执行高效地处理给定主题。请注意，asyncio 提供的是并发，而非并行。它通过使用事件循环在单个线程上实现并行，该事件循环会在某个任务空闲时（例如，等待网络请求）智能地切换到下一个任务。这营造出多个任务同时执行的效果，但代码本身仍然由单个线程执行，并受到 Python 全局解释器锁 (GIL) 的限制。

7

代码首先从 langchain_openai 和 langchain_core 导入必要的模块，包括语言模型、提示、输出解析和可运行结构等组件。代码尝试初始化一个 ChatOpenAI 实例，具体使用的是“gpt-4o-mini”模型，并指定了一个温度来控制创造力。在语言模型初始化过程中，使用了 try-except 块来增强鲁棒性。然后定义了三个独立的 LangChain“链”，每个链都针对输入主题执行不同的任务。第一个链用于简洁地概括主题，使用系统消息和包含主题占位符的用户消息。第二个链配置为生成三个与主题相关的有趣问题。第三个链设置为从输入主题中识别 5 到 10 个关键词，并要求它们以逗号分隔。每个独立的链都包含一个针对其特定任务定制的 ChatPromptTemplate，后面跟着已初始化的语言模型和一个用于将输出格式化为字符串的 StrOutputParser。

然后构建一个 RunnableParallel 模块，将这三个链捆绑在一起，使它们能够同时执行。这个并行可运行程序还包含一个 RunnablePassthrough，以确保原始输入主题可供后续步骤使用。为最终的综合步骤定义了一个单独的 ChatPromptTemplate，它以摘要、问题、关键词和原始主题作为输入，生成一个完整的答案。完整的端到端处理链，名为 full_parallel_chain，是通过将 map_chain（并行模块）按顺序插入综合提示符，然后是语言模型和输出解析器而创建的。提供了一个异步函数 run_parallel_example 来演示如何调用 full_parallel_chain。该函数以主题作为输入，并使用 invoke 来运行异步链。最后，标准的 Python if __name__ == "__main__": 模块展示了如何使用示例主题（在本例中为“太空探索的历史”）来执行 run_parallel_example，并使用 asyncio.run 来管理异步执行。

本质上，这段代码建立了一个工作流程，其中针对给定主题同时执行多个 LLM 调用（用于摘要、提问和术语检索），然后通过最终的 LLM 调用将它们的结果合并。这展示了使用 LangChain 在智能体工作流程中实现并行化的核心思想。

# 动手实践代码示例（Google ADK）

好的，现在让我们来看一个具体的例子，用 Google ADK 框架来阐释这些概念。我们将研究 ADK 的基本组件，例如：



ParallelAgent 和 SequentialAgent 可用于构建利用并发执行来提高效率的代理流程。

```
from google.adk.agents import LlmAgent,
ParallelAgent, SequentialAgent from google.adk.tools
import google_search GEMINI_MODEL="gemini-2.0-flash"

# --- 1. 定义研究员子代理（并行运行） ---

# 研究员 1：可再生能源
researcher_agent_1 = LlmAgent(
name="RenewableEnergyResearcher", model=GEMINI_MODEL,
instruction="""您是一位专注于能源领域的人工智能研究助理。请研究"可再生能源"
领域的最新进展。使用提供的谷歌搜索工具。请用简洁的语言（1-2句话）总结您的主要

发现。*仅*输出摘要。""",description="研究可再生能源。"
tools=[google_search], # 将结果存储在合并代理的状态

中 output_key="renewable_energy_result" )# 研究

员 2：电动汽车研究员_代理人_2 = LlmAgent(
name="EVResearcher", model=GEMINI_MODEL, instruction=""您是一位专注
于交通运输领域的人工智能研究助理。请研究"电动汽车技术"的最新发展。使用提供的谷

歌搜索工具。请用简洁的语言（1-2句话）总结您的主要发现。*仅*输出摘要。""",描

述="研究电动汽车技术。"
tools=[google_search], # 将结果存储在合并代理的状态

中 output_key="ev_technology_result" )# 研究员 3
```

：碳捕获研究员_代理_3 = LlmAgent(

9

name="CarbonCaptureResearcher", model=GEMINI_MODEL, instruction="""您是一位专注于气候解决方案的人工智能研究助理。请研究“碳捕获方法”的现状。使用提供的谷歌搜索工具。简明扼要地总结您的主要发现（1-2 句话）。*

仅*输出摘要。"""，description="研究碳捕获方法。"

tools=[google_search], # 将结果存储在合并代理的状态中

output_key="carbon_capture_result" )# --- 2. 创建并行代理（并行运行研究

人员） ---# 此代理协调研究人员的并发执行。# 当所有研究人员都完成任务并将结果存储在状态中后，该代理结束。parallel_research_agent = ParallelAgent( name="ParallelWebResearchAgent", sub_agents=[researcher_agent_1, researcher_agent_2, researcher_agent_3],

描述="并行运行多个研究代理以收集信息。"# --- 3. 定义合并代理（在并行代理*

之后*运行）--- # 此代理获取并行代理存储在会话状态中的结果，# 并将它们综合成一个带有归因的单一结构化响应。 merger_agent = LlmAgent(

name="SynthesisAgent", model=GEMINI_MODEL, # 如果需要，也可以使用更强大的模型进行合成

说明：您是一位人工智能助手，负责将研究成果整合为一份结构化报告。您的主要任务是综合以下研究摘要，并清晰地将研究成果归因于其来源领域。请使用标题组织您的报告，确保报告条理清晰、逻辑严谨，并能流畅地整合各个要点。

**至关重要：您的所有回复必须*完全*基于下方“输入摘要”中提供的信息。请勿添加任何其他内容。

1
0

任何未包含在本摘要中的外部知识、事实或细节。

**输入摘要：**

* **可再生能源：**

{renewable_energy_result} *

**电动汽车：**

{ev_technology_result} * **

碳捕获：**

碳捕获结果

**输出格式：**

## 近期可持续技术进展概述

### 可再生能源研究结果（基于 RenewableEnergyResearcher 的研究结果）[仅根据以上提供的可再生能源投入概要进行综合和详细阐述。]

### 电动汽车研究结果（基于 EVResearcher 的研究结果）[仅根据以上提供的电动汽车输入摘要进行综合和详细阐述。]

### 碳捕获研究结果（基于 CarbonCaptureResearcher 的研究结果）[仅根据以上提供的碳捕获输入摘要进行综合和详细阐述。]

### 总体结论 [请提供一段简短（1-2句话）的结论性陈述，该陈述仅需概括以上提出的研究结果。]

请*仅*按照此格式输出结构化报告。请勿在此结构之外添加任何引言或结论性短语，并严格遵守仅使用提供的输入摘要内容的原则。描述="将平行代理的研究成果结合起来一份结构严谨、引用详实、严格依据所提供的资料撰写的报告。
# 合并无需任何工具 # 此处无需 output_key，因为它的直接响应就是序列的最终输出）# --- 4. 创建 SequentialAgent（协调整体流程）



--- # 这是要运行的主代理。它首先执行 ParallelAgent 来填充状态，然后执行 MergerAgent 来生成最终输出。sequential_pipeline_agent = SequentialAgent( name="ResearchAndSynthesisPipeline", # 首先运行并行研究，然后合并 sub_agents=[parallel_research_agent, merger_agent], description="协调并行研究并综合结果。" )root_agent =

sequential_pipeline_agent

这段代码定义了一个多智能体系统，用于研究和整合可持续技术发展方面的信息。它设置了三个 LlmAgent 实例作为专门的研究员。ResearcherAgent_1 专注于可再生能源，ResearcherAgent_2 研究电动汽车技术，ResearcherAgent_3 研究碳捕获方法。每个研究员智能体都配置为使用 GEMINI_MODEL 模型和 google_search 工具。它们被指示用简

洁的语言（1-2 句话）总结研究结果，并使用 output_key 将这些摘要存储在会话状态中。

然后创建一个名为 ParallelWebResearchAgent 的并行代理，用于并发运行这三个研究人员代理。这样可以并行开展研究，从而节省时间。当所有子代理（即研究人员）都完成任务并填充状态后，并行代理的执行即告结束。

接下来，定义了一个合并代理（MergerAgent，也称 LlmAgent）来综合研究结果。该代理以并行研究人员存储在会话状态中的摘要作为输入。其指令强调输出必须严格基于提供的输入摘要，禁止添加外部知识。合并代理旨在将综合后的研究结果组织成一份报告，报告包含每个主题的标题以及一个简要的总体结论。

最后，创建一个名为 ResearchAndSynthesisPipeline 的 SequentialAgent 来协调整个工作流程。作为主控制器，该主代理首先执行 ParallelAgent 进行研究。ParallelAgent 完成后，SequentialAgent 再执行 MergerAgent 来综合收集到的信息。sequential_pipeline_agent 被设置为 root_agent，代表运行此多代理系统的入口点。整个过程如下。[12]
旨在高效地从多个来源并行收集信息，然后将其合并成一份结构化的报告。

# 概览

什么：许多智能体工作流包含多个子任务，这些子任务必须完成才能达成最终目标。纯粹的顺序执行，即每个任务都必须等待前一个任务完成，通常效率低下且速度缓慢。当任务依赖于外部 I/O 操作（例如调用不同的 API 或查询多个数据库）时，这种延迟会成为严重的瓶颈。如果没有并发执行机制，总处理时间就是所有单个任务持续时间的总和，这将影响系统的整体性能和响应速度。

为什么：并行化模式通过支持独立任务的并行执行，提供了一种标准化的解决方案。其工作原理是识别工作流中彼此不依赖直接输出的组件，例如工具使用或LLM调用。诸如LangChain和Google ADK之类的代理框架提供了内置结构来定义和管理这些并发操作。例如，主进程可以调用多个并行运行的子任务，并在所有子任务完成后再执行下一步。通过同时运行这些独立任务而不是依次运行，这种模式可以显著减少总执行时间。

经验法则：当工作流包含多个可以同时运行的独立操作时，可以使用这种模式，例如从多个 API 获取数据、处理不同的数据块或生成多个内容以供后续合成。

可视化摘要

1
3

图2:并行化设计模式

# 要点总结

以下是主要要点：

- 并行化是一种同时执行独立任务以提高效率的模式。
- 当任务涉及等待外部资源（例如 API 调用）时，它尤其有用。
- 采用并发或并行架构会带来相当大的复杂性和成本，影响设计、调试和系统日志记录等关键开发阶段。
- LangChain 和 Google ADK 等框架提供了对定义和管理并行执行的内置支持。
- 在 LangChain 表达式语言 (LCEL) 中，RunnableParallel 是并行运行多个可运行对象的关键构造。



- Google ADK 可以通过 LLM 驱动的委托来促进并行执行，其中协调器代理的 LLM 识别独立的子任务，并触发专门的子代理对它们的并发处理。
- 并行化有助于降低整体延迟，使代理系统能够更快速地响应复杂任务。

# 结论

并行化模式是一种通过并发执行独立的子任务来优化计算工作流程的方法。这种方法可以降低整体延迟，尤其是在涉及多个模型推理或调用外部服务的复杂操作中。

不同的框架提供了实现这种模式的不同机制。在 LangChain 中，像 RunnableParallel 这样的构造用于显式地定义和同时执行多个处理链。相比之下，像 Google Agent Developer Kit (ADK) 这样的框架可以通过多代理委托来实现并行化，其中主协调器模型将不同的子任务分配给可以并发运行的专用代理。

通过将并行处理与顺序（链接）和条件（路由）控制流相结合，可以构建复杂的、高性能的计算系统，从而有效地管理各种复杂的任务。

# 参考

以下是一些关于并行化模式及相关概念的深入阅读资源：

1. LangChain表达式语言（LCEL）文档（并行性）：

https://python.langchain.com/docs/concepts/lcel/2. Google Agent Developer Kit (ADK) 文档（多代理系统 ）：

https://google.github.io/adk-docs/agents/multi-agents/3. Python 异步这文档：

https://docs.python.org/3/library/asyncio.html15

# 第四章：反思

## 反射模式概述

在前几章中，我们探讨了基本的智能体模式：链式执行用于顺序执行，路由用于动态路径选择，并行化用于并发任务执行。这些模式使智能体能够更高效、更灵活地执行复杂任务。然而，即使拥有复杂的流程，智能体的初始输出或计划也可能并非最优、准确或完整。这就是……反射 模式开始发挥作用。

反思模式是指智能体评估自身的工作、输出或内部状态，并利用评估结果来改进自身表现或优化响应。这是一种自我纠正或自我改进的方式，使智能体能够根据反馈、内部评价或与预期标准的比较，迭代地优化输出或调整方法。有时，反思可以由一个独立的智能体来辅助完成，该智能体的特定职责是分析初始智能体的输出。

与简单的顺序链（输出直接传递到下一步）或路由（选择路径）不同，反射引入了一个反馈回路。智能体不仅产生输出，还会检查该输出（或生成该输出的过程），识别潜在问题或需要改进的地方，并利用这些洞察来生成更好的版本或调整其未来的行动。

该过程通常包括：

1.执行：代理执行任务或生成初始输出。2.评价/评论：代理人（通常使用另一个LLM调用或一组规则 ）

分析上一步的结果。此评估可能检查事实准确性、连贯性、风格、完整性、是否遵循指示或其他相关标准。3.反思/完善：根据批评意见，代理人决定如何

改进。这可能包括生成更精细的输出结果、调整后续步骤的参数，甚至修改整体方案。4.迭代（可选但常用）：然后可以执行改进后的输出或调整后的方法，并重复该反思过程，直到获得满意的结果或满足停止条件为止。

1

反思模式的一个关键且高效的实现方式是将过程拆分为两个截然不同的逻辑角色：生产者和评论者。这通常被称为“生成者-评论者”或“生产者-评论者”模型。虽然单个代理可以进行自我反思，但使用两个专门的代理（或使用不同的系统提示进行两次独立的LLM调用）通常能产生更稳健、更客观的结果。

1. 生产者代理：该代理的主要职责是执行任务的初始阶段。它专注于生成内容，无论是编写代码、撰写博客文章还是制定计划。它接收初始提示并生成输出的初始版本。

2. 评估代理：该代理的唯一目的是评估生产者生成的输出。它被赋予一套不同的指令，通常还有一个独特的角色（例如，“您是一位资深软件工程师”、“您是一位一丝不苟的事实核查员”）。评估代理的指令指导它根据特定标准分析生产者的工作，例如事实准确性、代码质量、风格要求或完整性。它的设计目的是发现缺陷、提出改进建议并提供结构化的反馈。

这种职责分离机制非常有效，因为它避免了智能体在审查自身作品时产生的“认知偏差”。评论智能体以全新的视角审视输出结果，专注于发现错误和改进之处。评论智能体的反馈随后会传递给生产智能体，生产智能体以此为指导，生成一个更加完善的新版本。提供的 LangChain 和 ADK 代码示例都实现了这种双智能体模型：LangChain 示例使用特定的“reflector_prompt”来创建评论智能体角色，而 ADK 示例则明确定义了生产智能体和评论智能体。

实现反射通常需要构建代理的工作流程，使其包含这些反馈循环。这可以通过代码中的迭代循环来实现，或者使用支持状态管理和基于评估结果的条件转换的框架。虽然可以在 LangChain/LangGraph、ADK 或 Crew.AI 链中实现单步评估和改进，但真正的迭代反射通常涉及更复杂的编排。

反思模式对于构建能够产出高质量输出、处理复杂任务并展现一定程度的自我意识和适应能力的智能体至关重要。它使智能体不再仅仅执行指令，而是转向更复杂的解决问题和内容生成方式。

2

反思与目标设定和监控（见第十一章）的交集值得关注。目标为智能体的自我评估提供了

最终基准，而监控则跟踪其进展。在许多实际案例中，反思可以作为纠正机制，利用监控反馈来分析偏差并调整策略。这种协同作用使智能体从被动的执行者转变为一个有目的的系统，能够自适应地实现其目标。

此外，当逻辑逻辑模型（LLM）保留对话记忆时（参见第8章），反思模式的有效性将显著提升。这种对话历史记录为评估阶段提供了至关重要的上下文，使智能体不仅能够孤立地评估其输出，还能将其置于先前的交互、用户反馈和不断变化的目标背景下进行评估。这使得智能体能够从过去的批评中学习，避免重蹈覆辙。如果没有记忆，每次反思都是一个独立的事件；有了记忆，反思就变成了一个累积过程，每个循环都建立在前一个循环的基础上，从而实现更智能、更具上下文感知能力的改进。

# 实际应用及案例

在输出质量、准确性或对复杂约束的遵守至关重要的场景中，反射模式非常有用：

1. 创意写作和内容创作：完善生成的文本、故事、诗歌或营销文案。
- 使用案例：一位经纪人正在撰写博客文章。
  - 反射：先写出草稿，然后根据流畅性、语气和清晰度进行点评，再根据点评进行修改。重复此过程，直到文章达到质量标准。
  - 益处：产出更精良、更有效的内容。

2. 代码生成和调试：编写代码、识别错误并修复错误。
- 使用案例：代理正在编写Python函数。
  - 反射：编写初始代码，运行测试或静态分析，找出错误或低效之处，然后根据发现的结果修改代码。
  - 益处：生成更健壮、更高效的代码。

3. 复杂问题解决：评估多步骤推理任务中的中间步骤或提出的解决方案。
- 使用案例：智能体正在解决逻辑谜题。



  - 反射：提出一个步骤，评估它是否更接近解决方案或引入矛盾，如果需要，回溯或选择不同的步骤。
  - 益处：提高智能体应对复杂问题空间的能力。

4. 总结和信息综合：完善摘要，使其准确、完整、简洁。

- 使用案例：代理人总结一份冗长的文件。
  - 反射：生成初步摘要，将其与原始文档中的关键点进行比较，完善摘要以包含缺失的信息或提高准确性。
  - 益处：生成更准确、更全面的摘要。

5. 规划和策略：评估拟议的计划，并找出潜在的缺陷或改进之处。

- 使用案例：代理人计划采取一系列行动来实现某个目标。
  - 反射：制定计划，模拟其执行情况或评估其在约束条件下的可行性，并根据评估结果修改计划。
  - 益处：制定更有效、更切合实际的计划。

6. 对话代理：回顾对话中的先前回合，以保持上下文、纠正误解或提高回复质量。

- 使用案例：客服聊天机器人。
  - 反射：在用户回复后，查看对话历史记录和最后生成的消息，以确保连贯性并准确回应用户的最新输入。
  - 益处：有助于展开更自然、更有效的对话。

反思为智能体系统增加了一层元认知能力，使它们能够从自身的输出和过程中学习，从而产生更智能、更可靠、更高质量的结果。

# 动手实践代码示例（LangChain）

实现完整的迭代反射过程需要状态管理和循环执行机制。虽然基于图的框架（例如 LangGraph）或自定义过程代码可以原生处理这些机制，但使用 LCEL（LangChain 表达式语言）的组合语法可以有效地演示单个反射循环的基本原理。

本示例使用 Langchain 库和 OpenAI 的 GPT-4o 模型实现反射循环，以迭代生成和改进计算以下参数的 Python 函数：[4]
计算一个数的阶乘。该过程从任务提示开始，生成初始代码，然后根据模拟高级软件工程师角色的反馈反复修改代码，每次迭代都对代码进行优化，直到反馈阶段认为代码完美或达到最大迭代次数。最后，程序会打印出最终优化后的代码。

首先，请确保已安装必要的库：

```
pip install langchain langchain-community
langchain-openai
```

您还需要使用您选择的语言模型（例如 OpenAI、Google Gemini、Anthropic）的 API 密钥来设置您的环境。

```
import os from dotenv import load_dotenv from
langchain_openai import ChatOpenAI from
langchain_core.prompts import ChatPromptTemplate from
langchain_core.messages import SystemMessage, HumanMessage

# --- 配置 --- # 从 .env 文件加载环境变量（用于 OPENAI_API_KEY）
load_dotenv()

# 检查是否已设置 API 密钥，如果未设置
os.getenv("OPENAI_API_KEY"):
  raise ValueError("在 .env 文件中找不到 OPENAI_API_KEY。请添加它。
")

# 初始化聊天逻辑学习模型 (LLM)。我们使用 gpt-4o 以获得更好的推理能
力。# 较低的温度用于获得更具确定性的输出。llm =
ChatOpenAI(model="gpt-4o", temperature=0.1)

def
run_reflection_loop():
演示多步骤 AI 反射循环，以逐步改进 Python 函数。
""" # --- 核心任务 --- task_prompt = """ 你的任务
是创建一个名为 `calculate_factorial` 的 Python 函
数。
   此函数应执行以下操作：1. 接受单个整数 `n` 作为
   输入。
```



```
2. 计算其阶乘 (n!)。3. 添加清晰的文档字符串，解释函数的功能。4. 处理特殊情
况：0 的阶乘为 1。5. 处理无效输入：如果输入为负数，则抛出 ValueError 异
常。
""" # --- 反射循环 --- max_iterations = 3 current_code = "" # 我们将
```

构建一个对话历史记录，以便在每一步中提供上下文。message_history =

[HumanMessage(content=task_prompt)]

for i in
range(max_iterations):

print("\n" + "="*25 + f" 反射循环：迭代 {i + 1} " + "="*25)#

--- 1. 生成/优化阶段 ---

# 第一次迭代生成结果，后续迭代进行改进。

如果 i ==
0：

print("\n>>> 第一阶段：生成初始代码...") # 第一条消息只是任务提示。

response = llm.invoke(message_history) current_code =

response.content else:print("\n>>> 第一阶段：根据先前的代码进行改进

评论……") # 消息历史记录现在包含该任务，

# 最后一段代码和最后一条评论。# 我们指示模型应用这些评论。
message_history.append(HumanMessage(content="请使用提供的评论改进代码。
"))

response =
llm.invoke(message_history)
current_code = response.content

print("\n--- 生成的代码 (v" + str(i + 1) + ") ---\n" +
current_code)

message_history.append(response) # 将生成的代码添加到历史记录中#

--- 2. 反思阶段 ---

print("\n>>> 第二阶段：反思生成的代码...")

# 为反射器代理创建特定提示。# 这会要求模型充当高级代码审查员的
角色。



反射器提示 = [

系统消息（内容=""
您是一位资深软件工程师，精通 Python。您的任务是进行细致的代码审查。请根据原始

任务要求，对提供的 Python 代码进行严格评估。查找代码中的错误、风格问题、缺失的边界情况以及需要改进的地方。如果代码完美且满足所有要求，请回复“CODE_IS_PERFECT”。否则，请提供一份包含所有批评意见的列表。"""），HumanMessage(content=f"原始任务：\n{task_prompt}\n\n待审查代码：\n{current_code}")

]critique_response =

llm.invoke(reflector_prompt)批评 =
Critique_response.content

# --- 3. 停止条件 --- 如果
critique 中显示“CODE_IS_PERFECT”
：
print("\n--- 评语 ---\n未发现其他评语。代码令人满意。")
休息

print("\n--- 评论 ---\n" + critique) # 将评论添加到历史记录中，以便进行下一次改进循环。
message_history.append(HumanMessage(content=f"对之前代码的评价：\n{critique}"))

print("\n" + "="*30 + " 最终结果 " + "="*30) print("\n反射过程后的最终精炼代码：\n") print(current_code)

如果 __name__ ==
"__main__":
运行反射循环()

代码首先设置环境，加载 API 密钥，并初始化一个强大的语言模型，例如 GPT-4o，并设置较低的温度以获得更精准的输出。核心任务由一个提示定义，该提示要求编写一个 Python 函数来计算一个数的阶乘，并包含对文档字符串、边界情况（例如 0 的阶乘）以及负数输入错误处理的具体要求。`run_reflection_loop` 函数负责协调迭代优化过程。在循环的第一次迭代中，7
语言模型根据任务提示生成初始代码。在后续迭代中，它会根据上一步的反馈意见对代码进行优化。语言模型还扮演着一个独立的“反馈器”角色，但使用不同的系统提示，它扮演着高级软件工程师的角色，根据原始任务需求对生成的代码进行评估。评估结果以问题列表的形式呈现，如果没有发现问题，则显示“代码完美”。循环持续进行，直到评估结果表明代码完美或达到最大迭代次数为止。对话历史记录会被维护，并在每个步骤中传递给语言模型，以便为生成/优化和反馈阶段提供上下文。最后，脚本会在循环结束后打印最后生成的

代码版本。

# 动手实践代码示例(ADK)

现在我们来看一个使用 Google ADK 实现的概念性代码示例。具体来说，该代码通过采用生成器-评论家结构来展示这一点，其中一个组件(生成器)生成初始结果或计划，而另一个组件(评论家)提供关键反馈或评论，引导生成器生成更完善或更准确的最终输出。

```
from google.adk.agents import SequentialAgent,
LlmAgent

# 第一个代理生成初始草稿。 generator = LlmAgent(
name="DraftWriter", description="生成给定主题的初始草稿内容。",
instruction="编写一段关于用户主题的简短、信息丰富的段落。"

  output_key="draft_text" # 输出结果保存到此状态键。)第二位经纪人对第一

位经纪人的草稿进行点评。审阅者 = LlmAgent(
name="事实核查器", description="审查给定文本的事实准确性并提供结构
化的评论。"
说明：您是一位一丝不苟的事实核查员。1. 阅读状态键“draft_text”中提供的文本。
2. 仔细核实所有声明的事实准确性。3. 您的最终输出必须是一个包含两个键的字典：-
“status”：一个字符串，值为“ACCURATE”或“INACCURATE”。- “reasoning”：一个
字符串，对您的状态提供清晰的解释，并引用发现的任何具体问题。
```



```
"", output_key="review_output" # 结构化字典保存在此处。 )#

SequentialAgent 确保生成器在审查器之前运行。review_pipeline =
SequentialAgent( name="WriteAndReview_Pipeline",

sub_agents=[generator, reviewer] )# 执行流程：# 1. 生成器运行 -> 将其段
落保存到 state['draft_text']。 # 2. 审阅者运行 -> 读取
state['draft_text'] 并将其字典输出保存到 state['review_output']。
```

这段代码演示了如何在 Google ADK 中使用顺序代理管道来生成和审核文本。它定义了两个 LlmAgent 实例：生成器和审核器。生成器代理用于生成给定主题的初始草稿段落。它被指示生成一段简短且信息丰富的文本，并将输出保存到状态键 draft_text 中。审核器代理充当生成器生成的文本的事实核查器。它被指示从 draft_text 读取文本并验证其事实准确

性。审核器的输出是一个结构化字典，包含两个键：status 和 reasoning。status 指示文本是“准确”还是“不准确”，而 reasoning 则提供对状态的解释。该字典保存到状态键 review_output 中。创建了一个名为 review_pipeline 的 SequentialAgent 来管理这两个代理的执行顺序。它确保生成器先运行，然后是审核器。整个执行流程是：生成器生成文本，然后将其保存到状态中。随后，审核员读取状态中的文本，进行事实核查，并将核查结果（状态和理由）保存回状态。这种流程允许使用不同的代理进行结构化的内容创建和审核。笔记：对于有兴趣的用户，我们还提供了一种使用 ADK LoopAgent 的替代实现方案。

在得出结论之前，需要注意的是，虽然反射模式能够显著提升输出质量，但它也存在一些重要的权衡取舍。迭代过程虽然强大，但会导致更高的成本和延迟，因为每次迭代循环都可能需要调用新的 LLM 函数，这使得它对于时间敏感型应用来说并非最佳选择。此外，该模式非常消耗内存；每次迭代都会扩展对话历史记录，包括初始输出、评论和后续的改进。



# 概览

什么：智能体的初始输出通常并非最优，可能存在不准确、不完整或无法满足复杂需求等问题。基本的智能体工作流程缺乏内置机制来识别和修复自身错误。解决这个问题的方法是让智能体评估自身的工作，或者更稳健地，引入一个独立的逻辑智能体作为评判者，从而防止初始响应不顾质量成为最终结果。

为什么：反射模式通过引入自我纠错和改进机制提供了一种解决方案。它建立了一个反馈回路：生产者代理生成输出，然后评论者代理（或生产者自身）根据预定义的标准对其进行评估。该评论随后用于生成改进版本。这种生成、评估和改进的迭代过程逐步提升最终结果的质量，从而获得更准确、更连贯、更可靠的结果。

经验法则：当最终输出的质量、准确性和细节比速度和成本更重要时，请使用反射模式。它尤其适用于生成精炼的长篇内容、编写和调试代码以及创建详细计划等任务。当任务需要高度客观性或专业评估，而通用型生产者代理可能忽略这些方面时，请使用单独的评论代理。

可视化摘要

图1:反射设计图案，自反射

1
1

图2：反思设计模式、生产者和评论者

# 要点总结

- 反射模式的主要优势在于它能够迭代地自我纠正和改进输出，从而显著提高质量、准确性和对复杂指令的遵守程度。
- 它包含执行、评估/评论和改进的反馈循环。反思对于需要高质量、准确或细致输出的任务至关重要。
- 生产者-评论家模型是一种强有力的实现方式，其中由一个独立的代理（或提示角色）来评估初始输出。这种职责分离增强了客观性，并允许提供更专业、更结构化的反馈。

- 然而，这些好处是以增加延迟和计算成本为代价的，同时还存在超出模型上下文窗口或被 API 服务限制的风险。
- 虽然完整的迭代反思通常需要有状态的工作流（如 LangGraph），但可以使用 LCEL 在 LangChain 中实现单个反思步骤，以传递输出进行评论和后续改进。
- Google ADK 可以通过顺序工作流程促进反思，其中一个代理的输出由另一个代理进行评判，从而允许后续的改进步骤。
- 这种模式使智能体能够进行自我纠正，并随着时间的推移提高其性能。

# 结论

反射模式为智能体工作流程中的自我纠错提供了一种关键机制，使其能够超越单次执行进行迭代改进。其实现方式是创建一个循环：系统生成输出，根据特定标准对其进行评估，然后利用该评估结果生成更优的结果。这种评估可以由智能体自身执行（自我反射），或者通常更有效的方式是由独立的评估智能体执行，这体现了该模式的一个关键架构选择。

虽然完全自主的多步骤反思过程需要强大的状态管理架构，但其核心原则可以通过单一的生成-批判-改进循环有效地展现出来。作为一种控制结构，反思可以与其他基础模式相结合，构建更强大、功能更复杂的智能体系统。

# 参考

以下是一些关于反射模式及相关概念的深入阅读资源：

1. 通过强化学习训练语言模型进行自我纠错，
https://arxiv.org/abs/2409.129172. LangChain表达式语言（LCEL）文档：
https://python.langchain.com/docs/introduction/3. LangGraph 文档：https://www.langchain.com/langgraph



4. Google Agent Developer Kit (ADK) 文档（多代理系统）：
https://google.github.io/adk-docs/agents/multi-agents/

# 第五章：工具使用（函数调用）

## 工具使用模式概述

到目前为止，我们讨论的代理模式主要涉及协调语言模型之间的交互以及管理代理内部工作流程中的信息流（链式调用、路由、并行化、反射）。然而，要使代理真正发挥作用并与现实世界或外部系统交互，它们需要具备使用工具的能力。

工具使用模式（通常通过称为函数调用的机制实现）使代理能够与外部 API、数据库、服务进行交互，甚至执行代码。它允许代理核心的 LLM 根据用户的请求或任务的当前状态来决定何时以及如何使用特定的外部函数。

该过程通常包括：

1.工具定义：外部功能或能力需向LLM进行定义和描述。该描述包括功能的用途、名称、接受的参数及其类型和描述。2.法学硕士录取决定：LLM接收用户的请求和可用的工具定义。LLM 根据对请求和工具的理解，决定是否需要调用一个或多个工具来完成请求。3.函数调用生成：如果LLM决定使用某个工具，它会生成一个从用户请求中提取的结构化输出（通常为 JSON 对象），其中指定要调用的工具的名称以及要传递给它的参数。4.工具执行：代理框架或编排层会拦截这种结构化输出。它会识别请求的工具，并使用提供的参数执行实际的外部函数。5.观察/结果：工具执行的输出或结果将返回到代理人。6.**LLM** 处理（可选但常用）：LLM 接收工具的输出作为上下文，并用它来形成对用户的最终回复，或决定工作流程中的下一步（这可能涉及调用另一个工具、反思或提供最终答案）。

这种模式至关重要，因为它突破了LLM训练数据的限制，使其能够访问最新信息、执行内部无法完成的计算、与用户特定数据交互或触发现实世界的操作。



调用是弥合 LLM 推理能力与各种可用外部功能之间差距的技术机制。

“函数调用”虽然恰当地描述了调用特定的、预定义的代码函数，但考虑“工具调用”这个更广泛的概念也很有意义。这个更宽泛的术语承认，代理的功能远不止于简单的函数执行。“工具”可以是传统的函数，也可以是复杂的API接口、数据库请求，甚至是指向另一个专用代理的指令。这种视角使我们能够设想更复杂的系统，例如，主代理可以将复杂的数据分

析任务委托给专门的“分析代理”，或者通过其API查询外部知识库。从“工具调用”的角度思考，能更好地体现代理在由数字资源和其他智能实体组成的多样化生态系统中作为协调者的全部潜力。

LangChain、LangGraph 和 Google Agent Developer Kit (ADK) 等框架为定义工具并将其集成到代理工作流程中提供了强大的支持，通常会利用现代 LLM（例如 Gemini 或 OpenAI 系列）的原生函数调用功能。在这些框架的“画布”上，您可以定义工具，然后配置代理（通常是 LLM 代理）以使其能够感知并使用这些工具。

工具使用是构建功能强大、交互式且具有外部感知能力的代理的基石模式。

# 实际应用及案例

工具使用模式几乎适用于任何需要代理超越生成文本来执行操作或检索特定动态信息的场景：

1. 从外部来源检索信息：访问 LLM 训练数据中不存在的实时数据或信息。
   - 使用案例：一位气象员。
     - 工具：一个天气 API，接受位置信息并返回当前天气状况。
     - 代理流程：用户询问“伦敦天气如何？”，LLM 识别出对天气工具的需求，调用该工具并输入“伦敦”，该工具返回数据，LLM 将数据格式化为用户友好的响应。
2. 与数据库和 API 交互：